



BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Bilgisayar Ağları Dersi – Dönem Projesi

**NetProbe: UDP Tabanlı Güvenilir Dosya Aktarımı,
Trafik İzleme ve Ağ Performans Analiz Platformu**

Grup 05

20360859046 Mustafa Can ERSOY

23360859729 Ahmet Furkan ÖCEL

24360859205 Murat DEMİRBAŞ

GitHub: https://github.com/AFurkanOcel/NetProbe_UDP_ReliableTransfer

Teslim Tarihi: [8.06.2026]

İçindekiler

İÇİNDEKİLER.....	5
1. Giriş.....	6
2. Problem Tanımı.....	6
2.1 UDP'nin Güvenilmezliği	6
2.2 Güvenilir Aktarım İçin Gerekli Mekanizmalar.....	6
2.3 Performans Analizi ve Karşılaştırma İhtiyacı.....	7
3. Sistem Mimarisi.....	8
3.1 Genel Mimari	8
3.2 Modül Yapısı.....	8
3.3 Veri Akışı.....	10
4. Protokol Tasarımı.....	10
4.1 Paket Formatları.....	10
4.2 Stop-and-Wait Protokolü	10
4.3 Go-Back-N (GBN) Protokolü	11
4.4 Bütünlük Doğrulama.....	12
5. Gerçekleme Detayları.....	13
5.1 Kullanılan Teknolojiler.....	13
5.2 Protokol Katmanı (protocol.py)	14
5.3 İstemci Gerçeklemesi (client.py).....	14
5.4 Sunucu Gerçeklemesi (server.py).....	16
5.5 Go-Back-N Gerçeklemesi (client_gbn.py / server_gbn.py).....	17
5.6 Loglama Sistemi (logger.py)	17
5.7 Analiz ve Grafik Üretimi (analysis.py)	18
6. Deney Ortamı.....	19
6.1 Donanım ve Yazılım Ortamı	19
6.2 Test Dosyaları	20
6.3 Deney Senaryoları.....	20
6.4 Yapay Kayıp Simülasyonu.....	20
7. Performans Metrikleri	22
7.1 Metrik Tanımları.....	22
7.2 Metrik Hesaplama Süreci	22
8. Sonuçlar ve Tartışma.....	23

8.1 Senaryo 1: Paket Boyutunun Etkisi.....	23
8.2 Senaryo 2: Timeout Değerinin Etkisi	25
8.3 Senaryo 3: Kayıp Oranının Etkisi	26
8.4 Senaryo 4: Dosya Boyutunun Etkisi.....	26
8.5 Bonus: Stop-and-Wait vs Go-Back-N Karşılaştırması.....	27
8.6 Bonus: UDP Reliable vs TCP Karşılaştırması	28
8.7 Genel Değerlendirme Tablosu.....	29
9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları	30
9.1 Paket Kaybı ve Timeout Yönetimi	30
9.2 Duplicate Paket Algılama	30
9.3 Büyük Dosya Aktarımında Bellek Yönetimi	31
9.4 Yapay Kayıp Simülasyonunun Tekrarlanabilirliği.....	31
9.5 TCP ile Karşılaştırmada Adil Koşulların Sağlanması	31
9.6 Ekran Çıktısının Log Dosyasına Yönlendirilmesi	32
10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler.....	32
10.1 Proje Sonuçları.....	32
10.2 Gelecekte Yapılabilecek Geliştirmeler.....	33
Kaynakça	34
Ek A: Proje Yapısı ve Dosya Listesi.....	35
Ek B: Çalıştırma Komutları	35
Ek C: Yer Tutucular Özeti (Doldurulacak Alanlar)	37
Çizelge 1: Paket Formatları.....	10
Çizelge 2: Örnek Log Dosyası Yapısı	18
Çizelge 3: Tüm Senaryolar Özet Tablosu	30
Şekil 1: UDP vs TCP protokol karşılaştırma diyagramı.....	7
Şekil 2: NetProbe Sistem Mimarisi Diyagramı.....	8
Şekil 3: Modül Bağımlılık Diyagramı	9
Şekil 4: Stop-and-Wait Zaman Diyagramı.....	11
Şekil 5: Go-Back-N Zaman Diyagramı.....	12
Şekil 6: İki Seviyeli Bütünlük Kontrolü Şeması	13
Şekil 7: İstemci Akış Şeması (Flowchart).....	15
Şekil 8: Sunucu Akış Şeması (Flowchart).....	16
Şekil 9: GBN Pencere Yönetimi Diyagramı.....	17

Şekil 10: Analiz Modülü İş Akış Diyagramı	19
Şekil 11: Deney Ortamı Şeması	21
Şekil 12: Metrik Hesaplama Süreci Akış Şeması	23
Şekil 13: S1: Throughput vs Paket Boyutu Grafiği	24
Şekil 14: S2: Completion Time vs Timeout Grafiği.....	25
Şekil 15: S3: Kayıp Oranının Etkisi Grafiği	26
Şekil 16: S4: Dosya Boyutunun Etkisi Grafiği.....	27
Şekil 17: SAW vs GBN Karşılaştırma Grafiği.....	28
Şekil 18: UDP Reliable vs TCP Karşılaştırma Grafiği.....	29

İÇİNDEKİLER

1. Giriş	3
2. Problem Tanımı	3
3. Sistem Mimarisi	4
4. Protokol Tasarımı	5
5. Gerçekleme Detayları	6
6. Deney Ortamı	7
7. Performans Metrikleri	8
8. Sonuçlar ve Tartışma	9
9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları	10
10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler	11
Kaynakça	12

1. Giriş

Bilgisayar ağı dersi kapsamında gerçekleştirilen bu proje, UDP (User Datagram Protocol) üzerinde uygulama katmanında güvenilir dosya aktarım mekanizmalarının tasarımı, geliştirilmesi ve performans analizini ele almaktadır. TCP (Transmission Control Protocol) gibi taşıma katmanında güvenilirlik sunan bir protokolün aksine, UDP bağlantısız ve güvenilir bir protokol olarak çalışmaktadır. Bu durum, UDP'nin düşük gecikme ve hafif yapı avantajlarını beraberinde getirirken, uygulama geliştiricilere güvenilirlik mekanizmalarını kendilerinin tasarlaması sorumluluğunu yükler.

NetProbe projesi, bu boşluğu doldurmak amacıyla UDP tabanlı bir istemci-sunucu mimarisi üzerinde stop-and-wait ve Go-Back-N (GBN) gibi güvenilir aktarım protokollerini uygulamaktadır. Proje kapsamında dosya aktarımı sırasında oluşan ağ olaylarının kayıt altına alınması, toplanan veriler üzerinden throughput, goodput, RTT, paket kayıp oranı ve yeniden gönderim oranı gibi performans metriklerinin ölçülmesi hedeflenmektedir. Ayrıca, geliştirilen UDP tabanlı sistem TCP ile karşılaştırmalı olarak değerlendirilmekte ve farklı ağ koşulları altındaki davranışı deneysel olarak incelenmektedir.

Bu rapor; projenin amacı, tasarım kararları, protokol yapısı, gerçekleştirme detayları, deney senaryoları, sonuçlar ve karşılaşılan teknik zorlukların çözüm yaklaşımlarını kapsamlı bir şekilde sunmaktadır.

2. Problem Tanımı

2.1 UDP'nin Güvenilmezliği

UDP, IP katmanı üzerinde çalışan ve taşıma katmanında herhangi bir güvenilirlik mekanizması sunmayan bir protokoldür. Paketler sıralı olarak iletilemeyebilir, kaybolabilir veya duplicate (tekrarlanmış) şekilde alıcıya ulaşabilir. Bu özellikler, gerçek zamanlı uygulamalar (VoIP, video streaming) için avantaj sağlarken, dosya aktarımı gibi veri bütünlüğünün kritik olduğu senaryolar için zorluk oluşturur.

2.2 Güvenilir Aktarım İçin Gerekli Mekanizmalar

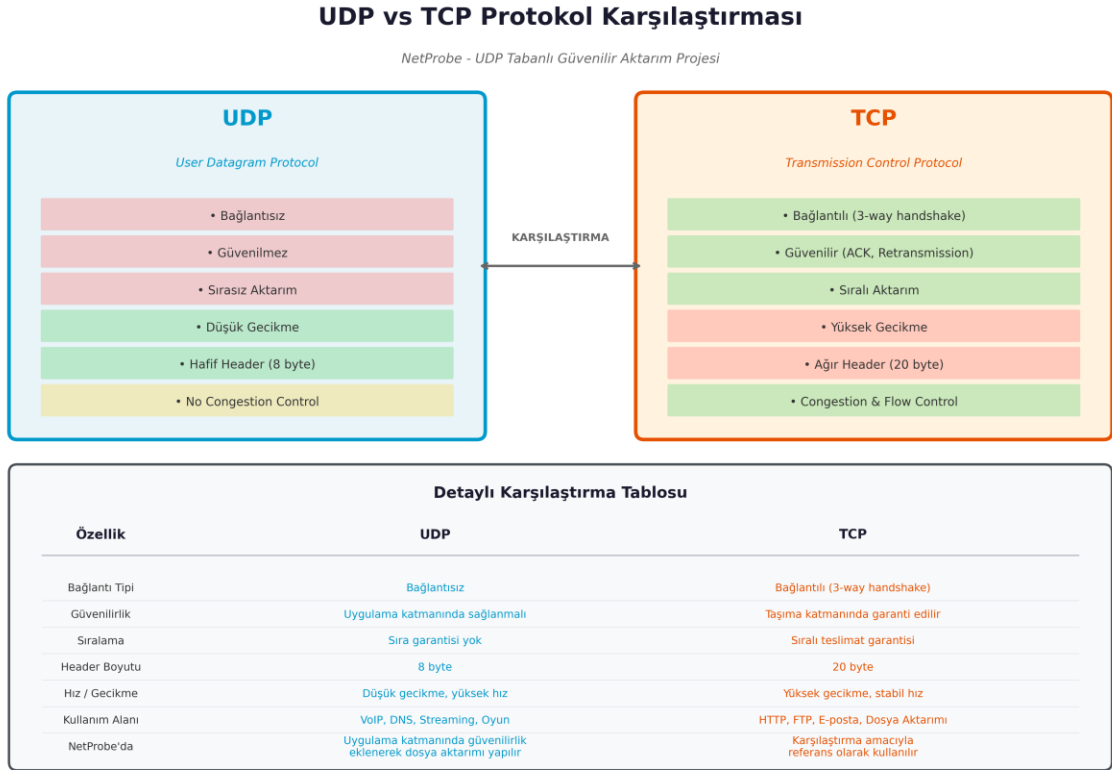
Güvenilir dosya aktarımı için aşağıdaki temel mekanizmaların uygulama katmanında gerçekleştirilmesi gerekmektedir:

- **Paket Numaralandırma (Sequence Number):** Gönderilen her veri paketine benzersiz bir sıra numarası atanarak, alıcının paketleri doğru sırada birleştirmesi sağlanır.
- **Onay Mesajları (ACK):** Alıcı, başarıyla aldığı her paket için göndericiye onay mesajı gönderir. Gönderici, ACK almadan bir sonraki paketi göndermez (stop-and-wait) veya pencere boyutunca gönderir (sliding window).
- **Zaman Aşımı Yönetimi (Timeout):** Belirli bir süre içinde ACK alınmazsa, paketin kaybolduğu varsayılır ve yeniden gönderilir.
- **Yeniden Gönderim (Retransmission):** Timeout sonrasında ilgili paketin tekrar iletilmesi sağlanır. Maksimum 5 deneme sınırı getirilmiştir.

- **Bütünlük Doğrulama (Checksum/Hash):** Paket bazlı SHA-256 hash ile payload bütünlüğü ve transfer sonunda dosya bazlı SHA-256 hash ile tam dosya doğrulaması yapılır.
- **Duplicate Paket Algılama:** Aynı sequence number'a sahip paketlerin tekrar dosyaya yazılması engellenir; uygun ACK tekrar gönderilir.

2.3 Performans Analizi ve Karşılaştırma İhtiyacı

Geliştirilen protokolün farklı ağ koşulları (paket boyutu, timeout değeri, kayıp oranı, dosya boyutu) altındaki davranışının anlaşılması ve TCP ile karşılaştırmalı olarak değerlendirilmesi, protokol tasarımının doğrulanması ve iyileştirilmesi açısından kritik öneme sahiptir.



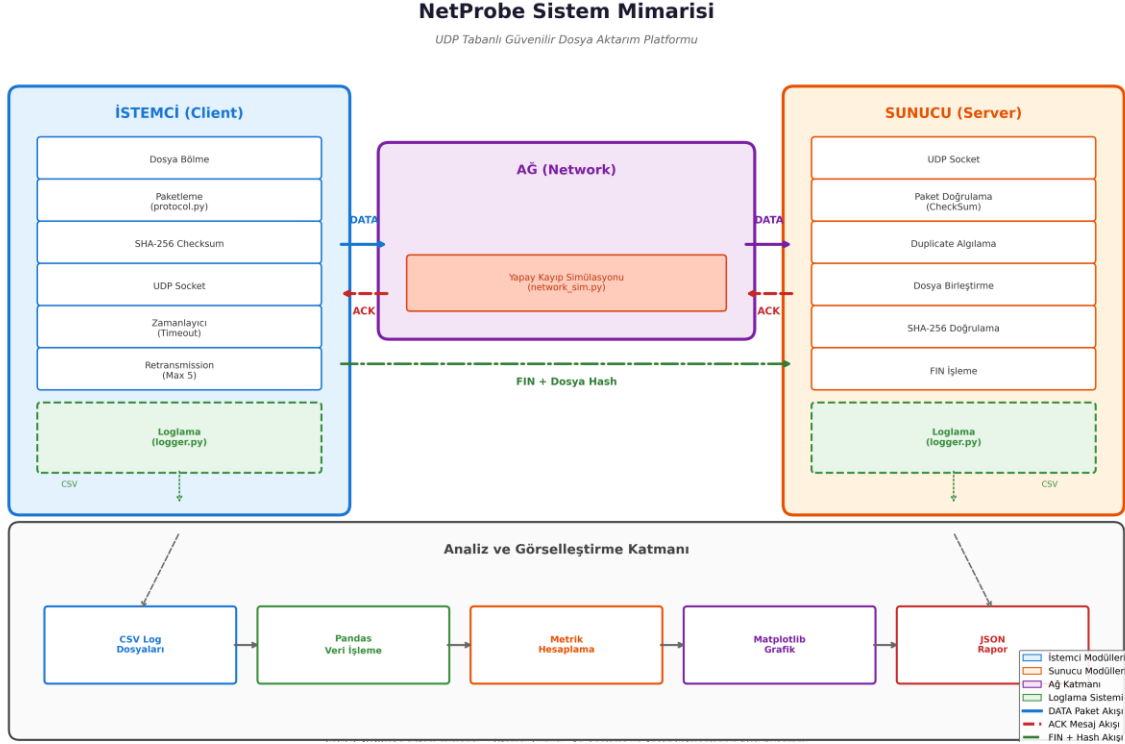
Şekil 1: UDP vs TCP protokol karşılaştırma diyagramı

(Bağlantılı vs Bağlantısız, Güvenilir vs Güvenilmez, Sıra)

3. Sistem Mimarisi

3.1 Genel Mimari

NetProbe sistemi, istemci-sunucu mimarisine dayanmaktadır. İstemci (client), aktarılabacak dosyayı parçalara böler ve UDP üzerinden sunucuya (server) gönderir. Sunucu, gelen paketleri doğrular, ACK mesajları üretir ve dosyayı yeniden birleştirir. Aktarım sonunda dosya bütünlüğü SHA-256 hash ile doğrulanır.



Şekil 2: NetProbe Sistem Mimarisi Diyagramı

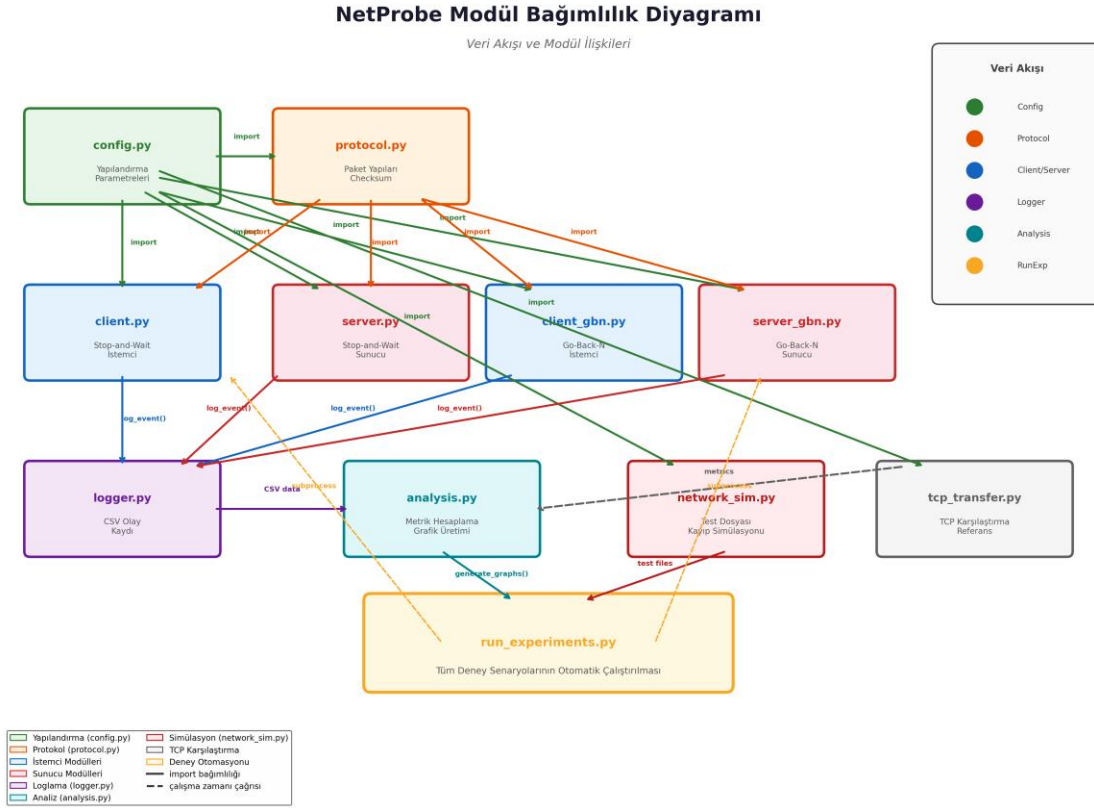
(İstemci → [Dosya Bölme → Paketleme → UDP Socket] → Ağ → [UDP Socket → Paket Doğrulama → Dosya Birleştirme] → Sunucu) Ayrıca ACK akışı (Sunucu → İstemci) ve loglama modüllerini gösteren blok diyagram.

3.2 Modül Yapısı

Proje aşağıdaki modüllerden oluşmaktadır:

- config.py:** Sistem parametrelerinin (paket boyutu, timeout, maksimum deneme sayısı, sunucu IP/port) merkezi olarak tanımlandığı yapılandırma dosyası.
- protocol.py:** DATA, ACK ve FIN paket yapılarının tanımlandığı, paket serileştirme/deserileştirme ve SHA-256 checksum hesaplama fonksiyonlarının bulunduğu protokol katmanı.

- **client.py / client_gbn.py**: Stop-and-Wait ve Go-Back-N aktarım mantığını uygulayan istemci modülleri.
- **server.py / server_gbn.py**: Stop-and-Wait ve Go-Back-N aktarım mantığını uygulayan sunucu modülleri.
- **logger.py**: Aktarım sırasında oluşan olayların (paket gönderim, ACK alma, timeout, yeniden gönderim) CSV formatında kaydedilmesini sağlayan loglama modülü.
- **analysis.py**: Toplanan log verilerinden throughput, goodput, RTT, loss rate, retry rate gibi metrikleri hesaplayan ve matplotlib ile grafik üreten analiz modülü.
- **network_sim.py**: Test dosyalarının oluşturulması ve yapay paket kayıp/gecikme simülasyonu için yardımcı fonksiyonlar.
- **run_experiments.py**: Tüm deney senaryolarının otomatik olarak çalıştırılması, metriklerin toplanması ve grafiklerin üretilmesini sağlayan otomasyon betiği.
- **tcp_transfer.py**: TCP tabanlı dosya aktarımı gerçekleştirerek UDP ile karşılaştırma yapılmasını sağlayan modül.



Şekil 3: Modül Bağımlılık Diyagramı

(config.py → protocol.py → client.py/server.py → logger.py → analysis.py → run_experiments.py akışını gösteren şema)

3.3 Veri Akışı

1. İstemci, dosyayı belirlenen paket boyutuna (örn. 1024 byte) göre parçalara böler.
2. Her parça için SHA-256 hash hesaplanır ve paket yapısına (packet type, sequence number, total packet count, payload length, checksum, payload) eklenir.
3. Paket UDP socket üzerinden sunucuya gönderilir.
4. Sunucu, gelen paketin checksum'ını doğrular; doğru ise ACK gönderir, yanlış ise sessizce atar (implicit NACK).
5. İstemci, ACK alırsa bir sonraki paketi gönderir; timeout olursa aynı paketi yeniden gönderir (max 5 deneme).
6. Tüm paketler başarıyla iletildiğinde, istemci FIN paketi gönderir.
7. Sunucu, tüm dosyanın SHA-256 hash'ini hesaplayarak bütünlük doğrulaması yapar.

4. Protokol Tasarımı

4.1 Paket Formatları

NetProbe, uygulama katmanında özel bir protokol tanımlamıştır. Üç temel paket türü bulunmaktadır: DATA, ACK ve FIN.

Paket Türü	Alanlar	Açıklama
DATA	packet_type, seq_no, total_count, payload_len, checksum, payload	Veri taşıyan ana paket
ACK	packet_type, ack_no, checksum	Onay mesajı
FIN	packet_type, total_hash	Aktarım sonu + dosya hash'i

Çizelge 1: Paket Formatları

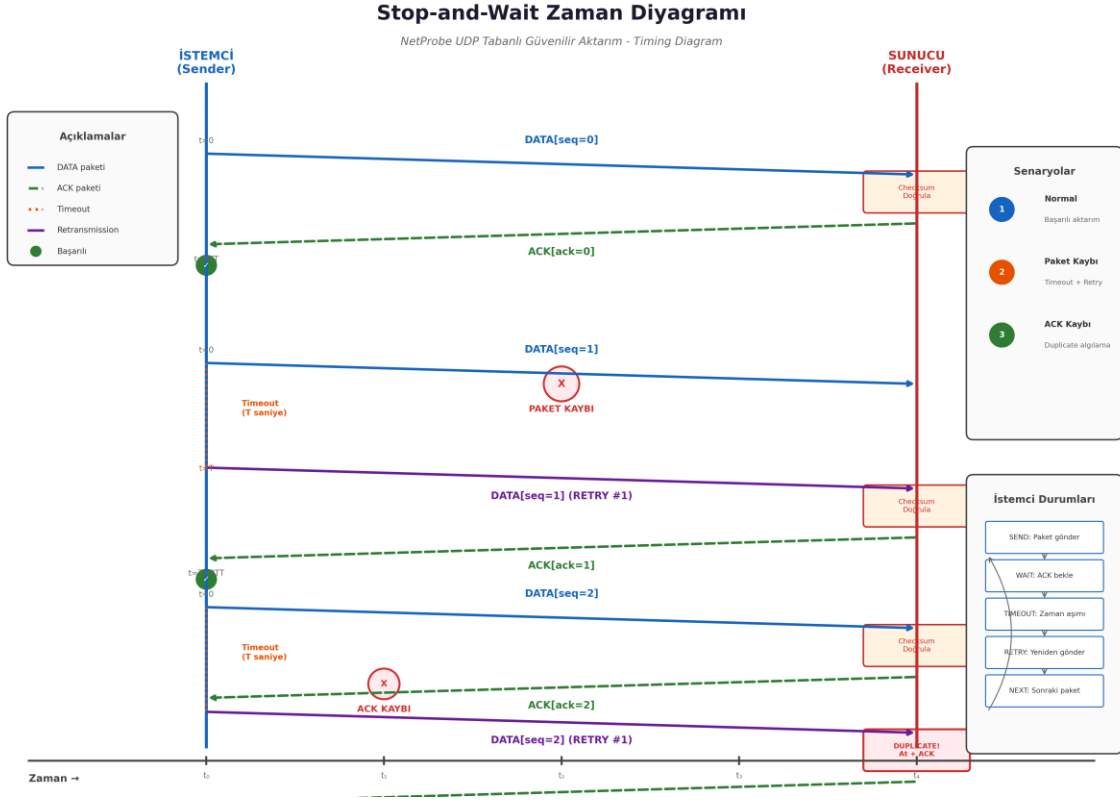
4.2 Stop-and-Wait Protokolü

Stop-and-Wait protokolünde, gönderici her seferinde tek bir paket gönderir ve ACK almadan bir sonraki paketi iletmez. Bu yöntem basit ve düşük bellek kullanımına sahiptir; ancak yüksek gecikmeli ağlarda verimlilik düşüktür.

Stop-and-Wait Algoritma Akışı:

1. seq_no = 0
2. Dosyayı paket boyutuna göre parçala
3. Her paket için:
 4. a. Paketi oluştur ve gönder
 5. b. Zamanlayıcıyı başlat (timeout = T)
 6. c. ACK bekle
7. - ACK gelirse: seq_no += 1, bir sonraki pakete geç
8. - Timeout olursa: retry_count += 1, paketi yeniden gönder

9. - `retry_count > MAX_RETRIES` ise: Transfer BAŞARISIZ
10. Tüm paketler tamamlandığında: FIN paketi gönder
11. Sunucu: Tüm dosya hash'ini hesapla ve doğrula



Şekil 4: Stop-and-Wait Zaman Diyagramı

(Timing Diagram) (Paket gönderim → ACK bekleme → Timeout → Retransmission senaryolarını gösteren sequence diagram)

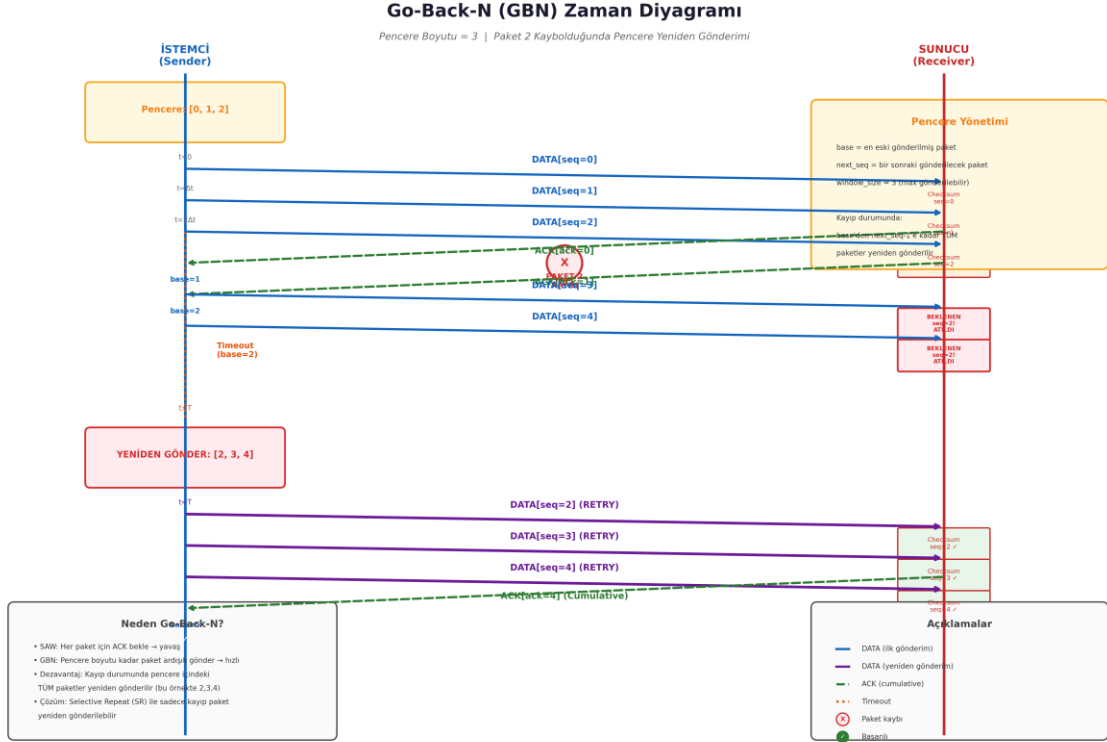
4.3 Go-Back-N (GBN) Protokolü

Go-Back-N, kayan pencere (sliding window) yaklaşımını kullanarak stop-and-wait'in verimlilik sorununu çözmeyi hedefler. Gönderici, pencere boyutu kadar paketi ACK beklemeden ardışık olarak gönderebilir. Eğer bir paketin ACK'si timeout süresi içinde gelmezse, o paketten itibaren pencere içindeki tüm paketler yeniden gönderilir.

GBN Algoritma Akışı:

12. `base = 0, next_seq = 0, window_size = N`
13. `next_seq < base + window_size` ve `next_seq < total_packets` ise:
14. a. Paketi gönder, `next_seq += 1`

15. Her gelen ACK için:
16. a. $base = ack_no + 1$ (pencere kaydır)
17. Timeout (base paket için):
18. a. $retry_count += 1$
19. b. base'den $next_seq-1$ 'e kadar tüm paketleri yeniden gönder
20. c. $retry_count > MAX_RETRIES$ ise: Transfer BAŞARISIZ
21. Tüm paketler ACK'lendiğinde: FIN gönder



Şekil 5: Go-Back-N Zaman Diyagramı

(Pencere boyutu = 3, Paket 2 kaybolduğunda Paket 2,3,4'ün yeniden gönderilmesini gösteren sequence diagram)

4.4 Bütünlük Doğrulama

NetProbe'da iki seviyeli bütünlük kontrolü uygulanmaktadır:

1. Paket Seviyesi: Her DATA paketinin payload'u için SHA-256 hash hesaplanır ve paket içinde checksum alanına yerleştirilir. Sunucu, gelen paketin hash'ini yeniden hesaplayarak karşılaştırır. Eşleşme olmazsa paket sessizce atılır (gönderici timeout sonrası yeniden gönderir).

2. Dosya Seviyesi: Tüm paketler başarıyla alındıktan sonra, sunucu alınan dosyanın tamamı

İki Seviyeli Bütünlük Kontrolü Şeması

NetProbe - SHA-256 Paket ve Dosya Seviyesi Doğrulama

SEVİYE 1: PAKET SEVİYESİ BÜTÜNLÜK KONTROLÜ

İSTEMCİ (Gönderici)

Dosya Parçası
 (Payload)

SHA-256
 Hash Hesaplama

DATA PAKETİ
 seq_no | payload_len |
 checksum (32B) | payload

UDP

SUNUCU (Alıcı)

GELEN PAKET
 seq_no | payload_len |
 checksum (32B) | payload

SHA-256 Doğrula
 checksum == hash(payload) ?

Neden İki Seviye?

Seviye 1:
Her paketin bozulmadığını doğrular

Seviye 2:
Tüm dosyanın eksiksiz ve doğru sırada alındığını doğrular

Tüm paketler başarıyla alındı

SEVİYE 2: DOSYA SEVİYESİ BÜTÜNLÜK KONTROLÜ

İSTEMCİ (Gönderici)

Tam Dosya
 (Tüm paketler birleştirilmeden önce)

SHA-256
 Dosya Hash'ı

FIN PAKETİ
 packet_type = 0x03
 total_hash (32 byte)

UDP

SUNUCU (Alıcı)

Birleştirilmiş Dosya
 (Tüm paketler dosyaya yazıldı)

SHA-256
 Dosya Hash'ı

FIN.total_hash == hash(dosya)?

EŞLEŞİYOR ✓

→ ACK gönder

EŞLEŞMİYOR ✗

→ Paket al. servisine bekle

SUNUCU (Alıcı) ?

İSTEMCİ (Gönderici)

Tam Dosya
 (Tüm paketler birleştirilmeden önce)

SHA-256
 Dosya Hash'ı

FIN PAKETİ
 packet_type = 0x03
 total_hash (32 byte)

UDP

SUNUCU (Alıcı)

Birleştirilmiş Dosya
 (Tüm paketler dosyaya yazıldı)

SHA-256
 Dosya Hash'ı

FIN.total_hash == hash(dosya)?

EŞLEŞİYOR ✓

→ ACK gönder

EŞLEŞMİYOR ✗

→ Paket al. servisine bekle

SUNUCU (Alıcı) ?

İSTEMCİ (Gönderici)

Tam Dosya
 (Tüm paketler birleştirilmeden önce)

SHA-256
 Dosya Hash'ı

FIN PAKETİ
 packet_type = 0x03
 total_hash (32 byte)

UDP

SUNUCU (Alıcı)

Birleştirilmiş Dosya
 (Tüm paketler dosyaya yazıldı)

SHA-256
 Dosya Hash'ı

FIN.total_hash == hash(dosya)?

EŞLEŞİYOR ✓

→ ACK gönder

EŞLEŞMİYOR ✗

→ Paket al. servisine bekle

SUNUCU (Alıcı) ?

İSTEMCİ (Gönderici)

Tam Dosya
 (Tüm paketler birleştirilmeden önce)

SHA-256
 Dosya Hash'ı

FIN PAKETİ
 packet_type = 0x03
 total_hash (32 byte)

UDP

SUNUCU (Alıcı)

Birleştirilmiş Dosya
 (Tüm paketler dosyaya yazıldı)

SHA-256
 Dosya Hash'ı

FIN.total_hash == hash(dosya)?

EŞLEŞİYOR ✓

→ ACK gönder

EŞLEŞMİYOR ✗

→ Paket al. servisine bekle

SUNUCU (Alıcı) ?

İSTEMCİ (Gönderici)

Tam Dosya
 (Tüm paketler birleştirilmeden önce)

SHA-256
 Dosya Hash'ı

FIN PAKETİ
 packet_type = 0x03
 total_hash (32 byte)

UDP

SUNUCU (Alıcı)

Birleştirilmiş Dosya
 (Tüm paketler dosyaya yazıldı)

SHA-256
 Dosya Hash'ı

FIN.total_hash == hash(dosya)?

EŞLEŞİYOR ✓

→ ACK gönder

EŞLEŞMİYOR ✗

→ Paket al. servisine bekle

SUNUCU (Alıcı) ?

İSTEMCİ (Gönderici)

Tam Dosya
 (Tüm paketler birleştirilmeden önce)

SHA-256
 Dosya Hash'ı

FIN PAKETİ
 packet_type = 0x03
 total_hash (32 byte)

UDP

SUNUCU (Alıcı)

Birleştirilmiş Dosya
 (Tüm paketler dosyaya yazıldı)

SHA-256
 Dosya Hash'ı

(Paket seviyesi SHA-256 → Dosya seviyesi SHA-256 doğrulama akışını gösteren diyagram)

5.1 Kullanılan Teknolojiler

Kütüphane/Modül	Kullanım Amacı	Proje İçindeki Rolü
socket	UDP/TCP soket programlama	İstemci-sunucu iletişimi
threading / asyncio	Eşzamanlılık	GBN'de zamanlayıcı ve ACK dinleme
hashlib (SHA-256)	Kriptografik hash	Paket ve dosya bütünlük kontrolü
csv	Veri kaydı	Olay loglarının CSV

		formatında saklanması
json	Veri serileştirme	Deney metriklerinin JSON olarak saklanması
matplotlib	Grafik çizimi	Performans metriklerinin görselleştirilmesi
pandas	Veri analizi	Log verilerinin işlenmesi ve analizi
time	Zaman ölçümü	RTT, throughput ve completion time hesaplamaları

5.2 Protokol Katmanı (protocol.py)

protocol.py modülü, uygulama katmanı protokolünün temelini oluşturur. Paket yapıları, serileştirme/deserileştirme fonksiyonları ve SHA-256 checksum hesaplama fonksiyonları bu modülde tanımlanmıştır. Paketler, Python struct modülü kullanılarak binary formata dönüştürülür ve UDP üzerinden iletilir.

Paket Serileştirme Örneği (Sözde Kod):

```
# DATA Paketi Yapısı
struct_format = "!B I I I 32s" # type(1B), seq_no(4B), total(4B),
                                # payload_len(4B), checksum(32B)

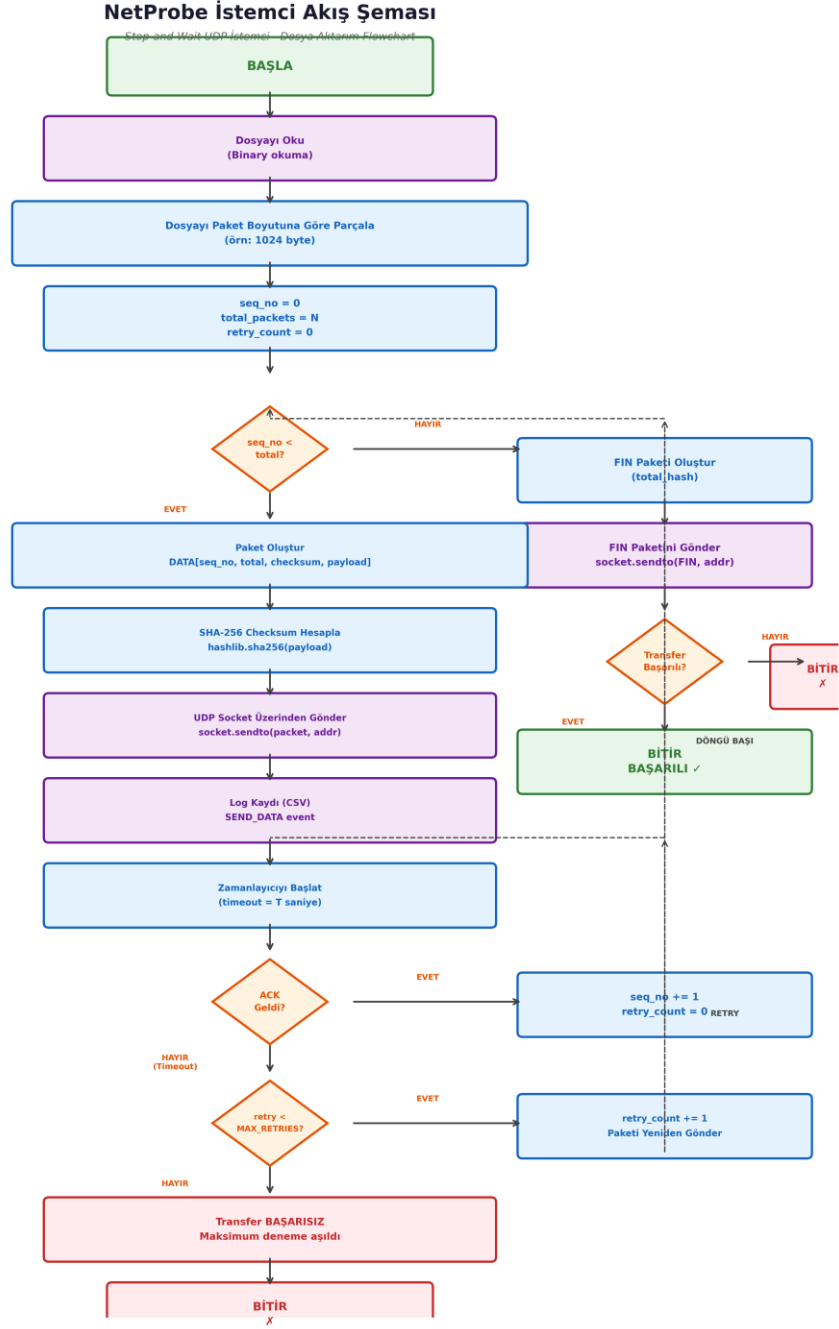
# ACK Paketi Yapısı
struct_format = "!B I 32s"      # type(1B), ack_no(4B), checksum(32B)

# SHA-256 Checksum Hesaplama
def compute_checksum(payload: bytes) -> bytes:
    return hashlib.sha256(payload).digest()

# Paket Doğrulama
def verify_packet(packet) -> bool:
    return compute_checksum(packet.payload) == packet.checksum
```

5.3 İstemci Gerçeklemesi (client.py)

Stop-and-Wait istemci, dosyayı parçalara böler, her parça için paket oluşturur ve UDP üzerinden gönderir. Her paket gönderiminde zamanlayıcı başlatılır ve ACK bekleme döngüsüne girilir. Timeout durumunda aynı paket yeniden gönderilir (maksimum 5 deneme). Tüm paketler başarıyla iletilirse FIN paketi gönderilir ve transfer durumu kullanıcıya bildirilir.

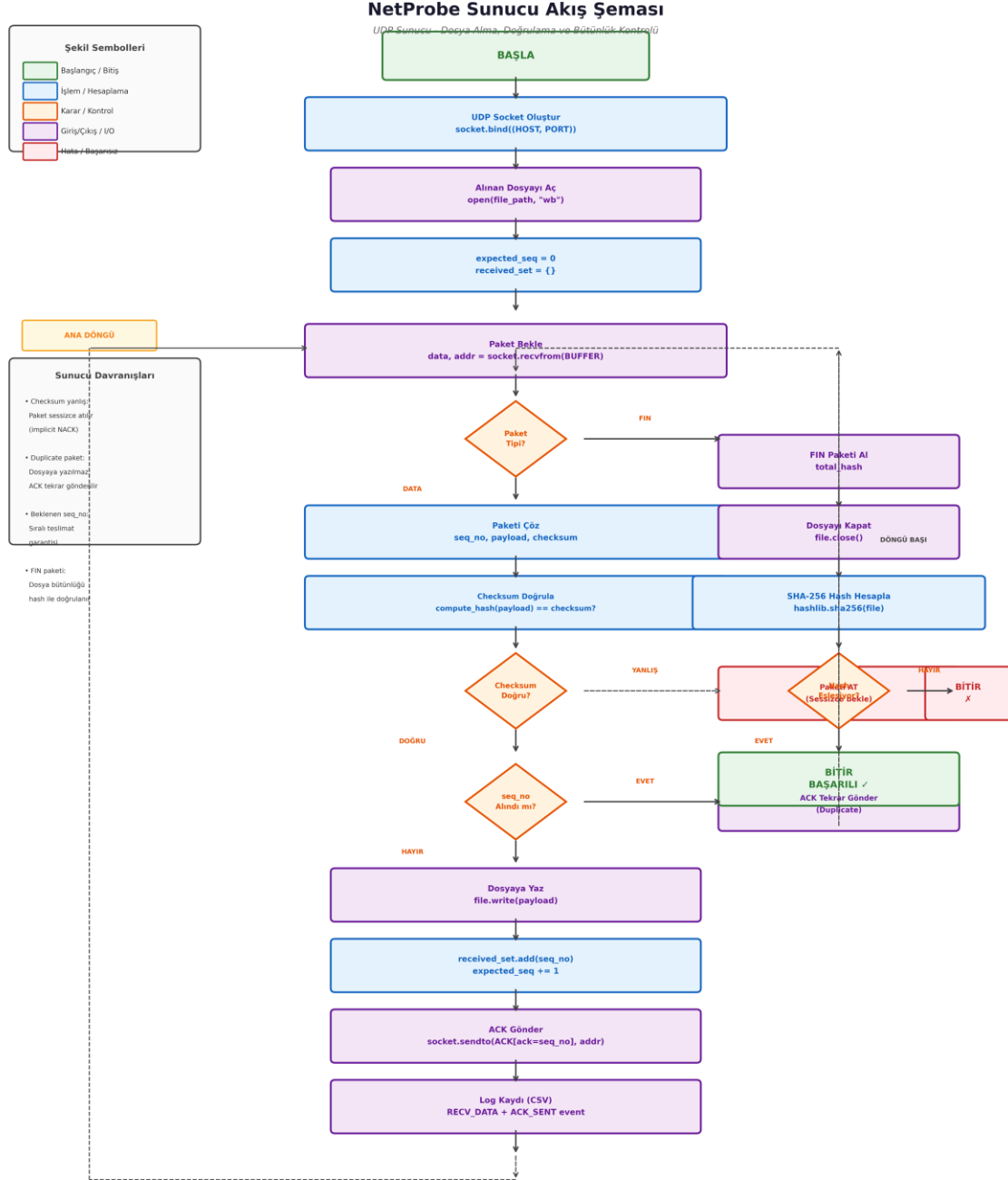


Şekil 7: İstemci Akış Şeması (Flowchart)

(Dosya okuma → Paketleme → Gönderim → ACK bekleme → Timeout kontrolü → Retransmission → FIN gönderim akışını gösteren akış şeması)

5.4 Sunucu Gerçeklemesi (server.py)

Sunucu, belirtilen UDP portunu dinler ve gelen paketleri işler. DATA paketleri için checksum doğrulaması yapar, doğru paketler için ACK gönderir ve payload'u dosyaya yazar. Duplicate paketler (aynı sequence number) algılanır, dosyaya tekrar yazılmaz ancak ACK tekrar gönderilir. FIN paketi alındığında dosya bütünlüğü SHA-256 ile doğrulanır.

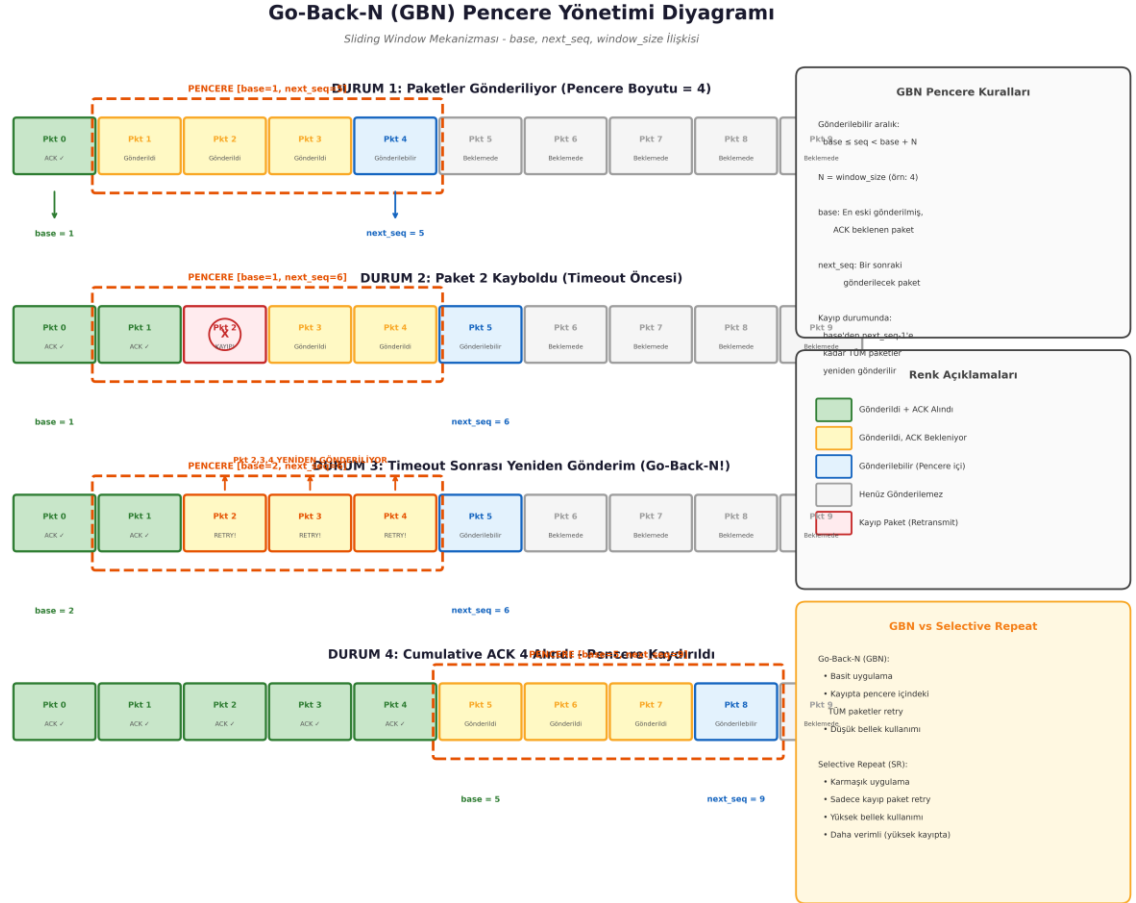


Şekil 8: Sunucu Akış Şeması (Flowchart)

(Port dinleme → Paket alma → Checksum doğrulama → ACK gönderim → Dosya yazma → FIN kontrolü → Hash doğrulama akışını gösteren akış şeması)

5.5 Go-Back-N Gerçeklemesi (client_gbn.py / server_gbn.py)

GBN gerçeeklemesinde, gönderici pencere boyutu kadar paketi ardışık olarak gönderir. Her paket için ayrı bir zamanlayıcı yerine, pencerenin en eski paketi (base) için tek bir zamanlayıcı kullanılır. ACK geldiğinde pencere ilerletilir ($base = ack_no + 1$). Timeout olduğunda base'den next_seq-1'e kadar tüm paketler yeniden gönderilir. Sunucu tarafında, beklenen sequence number dışındaki paketler sessizce atılır (cumulative ACK).



Şekil 9: GBN Pencere Yönetimi Diyagramı

(base, next_seq, window_size arasındaki ilişkiyi ve pencere kaydırma mekanizmasını gösteren şema)

5.6 Loglama Sistemi (logger.py)

logger.py modülü, hem istemci hem de sunucu tarafında CSV formatında olay kaydı tutar. Kaydedilen olaylar şunlardır:

- Paket gönderim zamanı ve sequence number

- ACK alınma zamanı ve ack number
- Timeout oluşumu ve ilgili sequence number
- Yeniden gönderim sayısı ve nedeni
- Transfer başlangıç/bitiş zamanları
- Başarılı ve başarısız paket sayıları

	A	B	C	D	E	F	G	H	I		A	B	C	D	E	F	G	H
1	timestamp	event_type	seq_num	retry_count	elapsed_ms	notes					1	timestamp	event_type	seq_num	retry_count	elapsed_ms	notes	
2	1779742049.322349	[CLIENT] TRANSFER_START	-1,0,-1.0	file=test_10KB.bin size=10240B packets=40							2	1779742049.3228514	[SERVER] DATA_RECEIVED	0,0,-1.0				
3	1779742049.3228514	[CLIENT] SENT	0,0,-1.0								3	1779742049.3238597	[SERVER] DATA_RECEIVED	1,0,-1.0				
4	1779742049.3238597	[CLIENT] ACK_RECEIVED	0,0,1.008								4	1779742049.3249536	[SERVER] DATA_RECEIVED	2,0,-1.0				
5	1779742049.3238597	[CLIENT] SENT	1,0,-1.0								5	1779742049.3256652	[SERVER] DATA_RECEIVED	3,0,-1.0				
6	1779742049.3246198	[CLIENT] ACK_RECEIVED	1,0,0.0								6	1779742049.3260345	[SERVER] DATA_RECEIVED	4,0,-1.0				
7	1779742049.3249536	[CLIENT] SENT	2,0,-1.0								7	1779742049.3278787	[SERVER] DATA_RECEIVED	5,0,-1.0				
8	1779742049.3249536	[CLIENT] ACK_RECEIVED	2,0,0.0								8	1779742049.3278787	[SERVER] DATA_RECEIVED	6,0,-1.0				
9	1779742049.3256652	[CLIENT] SENT	3,0,-1.0								9	1779742049.328887	[SERVER] DATA_RECEIVED	7,0,-1.0				
10	1779742049.3260345	[CLIENT] ACK_RECEIVED	3,0,0.14								10	1779742049.3296132	[SERVER] DATA_RECEIVED	8,0,-1.0				
11	1779742049.3260345	[CLIENT] SENT	4,0,-1.0								11	1779742049.3298032	[SERVER] DATA_RECEIVED	9,0,-1.0				
12	1779742049.3265107	[CLIENT] ACK_RECEIVED	4,0,0.0								12	1779742049.3298032	[SERVER] DATA_RECEIVED	10,0,-1.0				
13	1779742049.3278787	[CLIENT] SENT	5,0,-1.0								13	1779742049.331167	[SERVER] DATA_RECEIVED	11,0,-1.0				
14	1779742049.3278787	[CLIENT] ACK_RECEIVED	5,0,0.0								14	1779742049.331167	[SERVER] DATA_RECEIVED	12,0,-1.0				
< > transfer_log_client +										< > transfer_log_server +								

Çizelge 2: Örnek Log Dosyası Yapısı

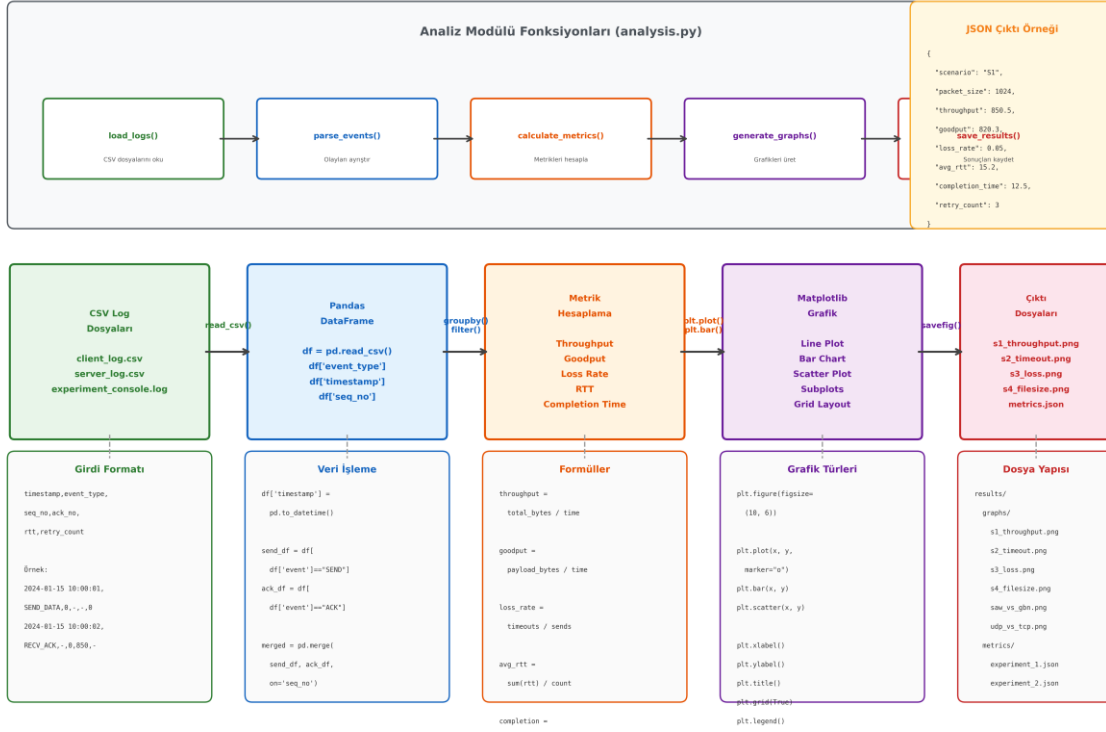
5.7 Analiz ve Grafik Üretimi (analysis.py)

analysis.py modülü, CSV log dosyalarını okuyarak performans metriklerini hesaplar ve matplotlib kütüphanesi ile görselleştirir. Hesaplanan metrikler:

- **Throughput:** Toplam gönderilen byte (retransmission dahil) / toplam aktarım süresi (byte/saniye)
- **Goodput:** Başarıyla teslim edilen payload byte / toplam aktarım süresi (byte/saniye)
- **Packet Loss Rate:** Timeout sayısı / toplam gönderim sayısı $\times 100$
- **Retransmission Rate:** Yeniden gönderim sayısı / toplam paket sayısı $\times 100$
- **Average RTT:** Başarıyla alınan ACK'lerde ölçülen ortalama gidiş-dönüş süresi (ms)
- **Completion Time:** Transferin başlangıcından bitişine kadar geçen toplam süre (saniye)

NetProbe Analiz Modülü İş Akış Diyagramı

Log Verilerinden Performans Grafiklerine - analysis.py İş Akışı



Şekil 10: Analiz Modülü İş Akış Diyagramı

(CSV Log → Pandas DataFrame → Metrik Hesaplama → Matplotlib Grafik → PNG/JPEG Çıktı akışını gösteren şema)

6. Deney Ortamı

6.1 Donanım ve Yazılım Ortamı

Deneyler aşağıdaki ortamda gerçekleştirilmiştir:

Parametre	Değer
İşletim Sistemi	[Windows 11 / Ubuntu 22.04 / macOS - KENDİ SİSTEMİNİZİ YAZIN]
İşlemci	[Örn: Intel Core i5-12400 / AMD Ryzen 5 5600X]
RAM	[Örn: 16 GB DDR4]
Python Sürümü	3.10+
Ağ Ortamı	[Yerel makine (localhost) / LAN / Laboratuvar ağı]
Sunucu IP	127.0.0.1 (localhost)
Sunucu Port	[config.py'deki port numarası, örn: 5005]

6.2 Test Dosyaları

Deneylerde kullanılan test dosyaları network_sim.py betiği ile rastgele byte dizileri olarak üretilmiştir. Dosya boyutları ve amaçları aşağıdaki tabloda özetlenmiştir:

Dosya Adı	Boyut	Kullanım Amacı
test_10KB.bin	10 KB	Küçük dosya aktarımı performansı
test_100KB.bin	100 KB	Orta ölçekli dosya aktarımı
test_1MB.bin	1 MB	Büyük dosya aktarımı performansı
test_10MB.bin	10 MB	Çok büyük dosya aktarımı ve ölçeklenebilirlik

6.3 Deney Senaryoları

Proje kapsamında 4 zorunlu deney senaryosu ve 2 bonus karşılaştırma senaryosu uygulanmıştır. Her veri noktası N_REPEATS = 3 tekrar ile çalıştırılmış ve ortalama değerler raporlanmıştır.

Senaryo	Değişen Parametre	Sabit Parametreler	Amaç
S1: Paket Boyutu	256, 512, 1024, 4096 byte	Timeout=2s, Loss=0%	Paket boyutunun throughput ve completion time üzerindeki etkisi
S2: Timeout Değeri	0.5, 1.0, 2.0, 5.0 saniye	PacketSize=1024, Loss=5%	Timeout değerinin gereksiz retransmission ve gecikme üzerindeki etkisi
S3: Kayıp Oranı	0%, 5%, 15%, 30%	PacketSize=1024, Timeout=2s	Simüle edilmiş paket kaybı koşullarında sistem davranışı
S4: Dosya Boyutu	10KB, 100KB, 1MB, 10MB	PacketSize=1024, Timeout=2s, Loss=0%	Küçük ve büyük dosya aktarımında sistem verimliliği
Bonus: SAW vs GBN	Stop-and-Wait vs Go-Back-N	PacketSize=1024, Timeout=2s	İki protokolün performans karşılaştırması
Bonus: UDP vs TCP	UDP Reliable vs TCP	PacketSize=1024	UDP tabanlı güvenilir aktarım ile TCP karşılaştırması

6.4 Yapay Kayıp Simülasyonu

Paket kaybı simülasyonu, network_sim.py modülünde rastgele sayı üretici kullanılarak gerçekleştirilir. Belirlenen kayıp oranına göre (örn. %5), her gönderilen paket için rastgele

bir karar verilir ve belirli olasılıkla paket düşürülür (gönderilmez). Bu yöntem, gerçek ağ koşullarını laboratuvar ortamında tekrarlanabilir şekilde simüle etmeyi sağlar.

Yapay Kayıp Algoritması (Sözde Kod):

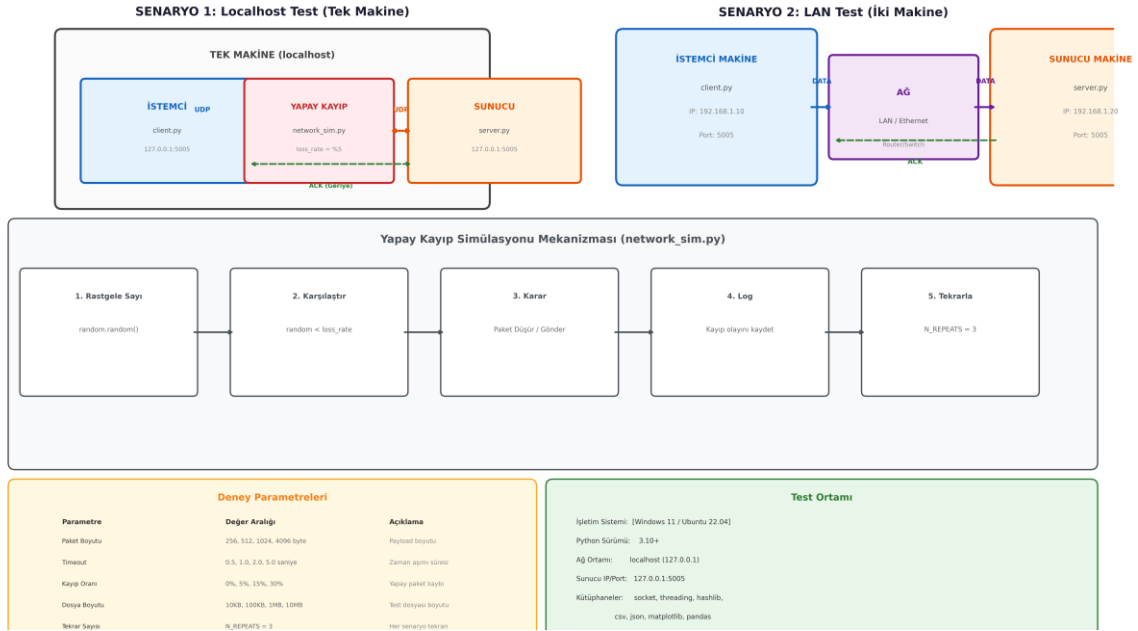
```
import random

def simulate_loss(loss_rate_percent):
    return random.random() < (loss_rate_percent / 100)

# Kullanım:
if not simulate_loss(LOSS_RATE):
    udp_socket.sendto(packet, address) # Paket gönder
else:
    pass # Paket düşür (simüle edilmiş kayıp)
```

NetProbe Deney Ortamı Şeması

UDP Tabanlı Güvenilir Aktarım - Ağ Topolojisi ve Test Düzeni



Şekil 11: Deney Ortamı Şeması

(İstemci makinesi → [UDP Socket + Yapay Kayıp Modülü] → Ağ → [UDP Socket] → Sunucu makinesi)

7. Performans Metrikleri

7.1 Metrik Tanımları

Bu bölümde, deneylerde kullanılan performans metriklerinin matematiksel tanımları ve teknik açıklamaları sunulmaktadır:

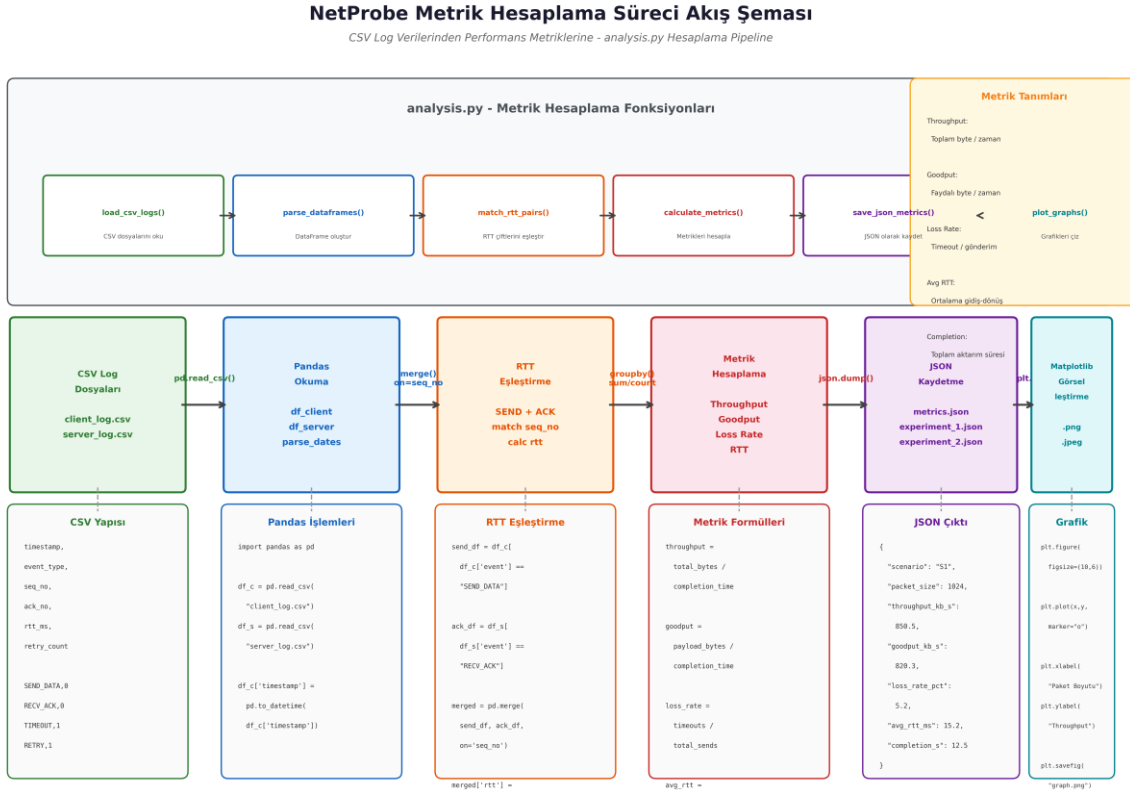
Metrik	Formül	Açıklama
Throughput	$\text{Toplam Gönderilen Byte} / \text{Completion Time}$	Aktarım süresince ağdan geçen toplam veri miktarı (retransmission dahil). Ağın ham kapasite kullanımını gösterir.
Goodput	$\text{Başarıyla Teslim Edilen Payload} / \text{Completion Time}$	Gerçekten faydalı olan veri aktarım hızı. Retransmission ve overhead hariç, uygulama katmanındaki etkin veri hızını ölçer.
Packet Loss Rate	$\text{Timeout Sayısı} / \text{Toplam Gönderim Sayısı} \times 100$	Paketlerin kaybolduğu oran. Yüksek değerler ağ kalitesinin düşük olduğunu gösterir.
Retransmission Rate	$\text{Yeniden Gönderim Sayısı} / \text{Toplam Paket Sayısı} \times 100$	Aynı paketin kaç kez yeniden gönderildiğini gösterir. Protokol verimliliğinin önemli bir göstergesidir.
Average RTT	$\Sigma(\text{ACK Zamanı} - \text{Gönderim Zamanı}) / \text{Başarılı Paket Sayısı}$	Paketin gönderilmesinden ACK alınmasına kadar geçen ortalama süre. Ağ gecikmesinin temel ölçüsüdür.
Completion Time	Transfer Bitiş - Transfer Başlangıç	Tüm dosyanın aktarımının tamamlanması için geçen toplam süre. Kullanıcı deneyimi açısından kritik metriktir.

7.2 Metrik Hesaplama Süreci

analysis.py modülünde metrik hesaplama süreci şu adımları izler:

1. CSV log dosyaları pandas DataFrame olarak yüklenir.
2. Gönderim ve ACK olayları eşleştirilerek RTT değerleri hesaplanır.
3. Timeout ve retry olayları sayılarak loss rate ve retransmission rate belirlenir.
4. Toplam byte sayısı (header + payload) ve başarılı payload byte sayısı ayrı ayrı toplanır.

- Completion time, ilk gönderim ile son ACK/FIN arasındaki fark olarak hesaplanır.
- Tüm metrikler JSON formatında kaydedilir ve matplotlib ile görselleştirilir.



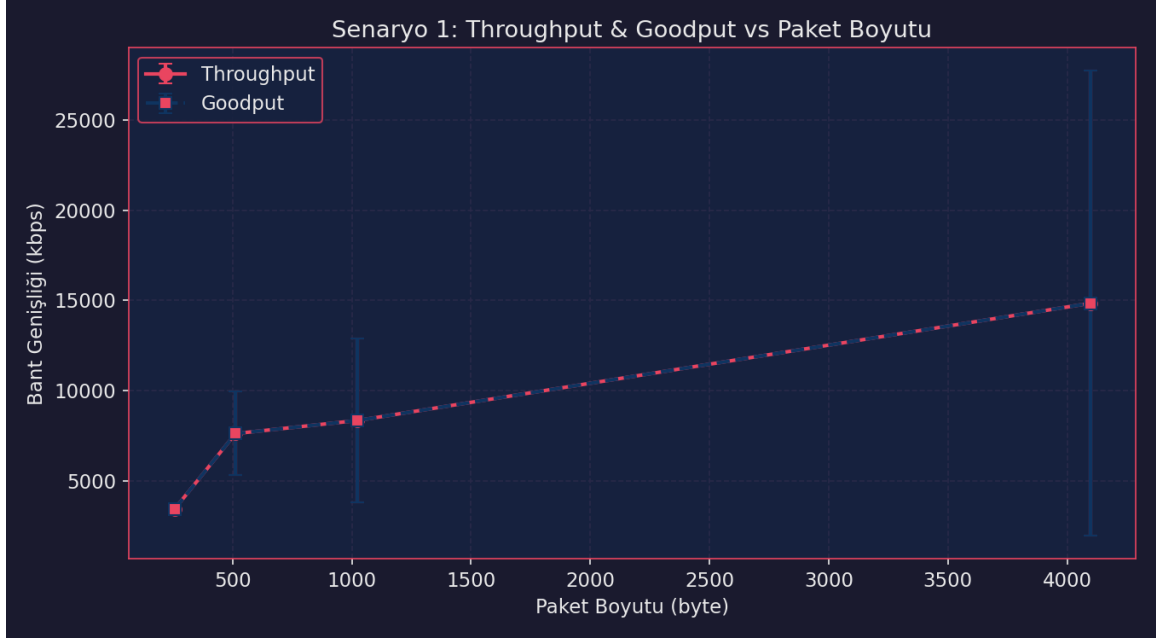
Şekil 12: Metrik Hesaplama Süreci Akış Şeması

(CSV Log → Pandas Okuma → RTT Eşleştirme → Metrik Hesaplama → JSON Kaydetme → Matplotlib Görselleştirme)

8. Sonuçlar ve Tartışma

8.1 Senaryo 1: Paket Boyutunun Etkisi

Bu senaryoda, 256, 512, 1024 ve 4096 byte paket boyutları kullanılarak throughput ve completion time metrikleri ölçülmüştür. Timeout=2s ve kayıp oranı=0% sabit tutulmuştur.



Şekil 13: S1: Throughput vs Paket Boyutu Grafięi

Teknik Yorum:

Küçük paket boyutları (250 byte): Paket boyutu küçüldükçe protokol başlıklarının (header) toplam veriye oranı artar. Bu durum verimlilięi düşürdüęü için hem Throughput hem de Goodput deęerlerini **en düşük seviyede (yaklaşık 3400 kbps)** bırakmıřtır.

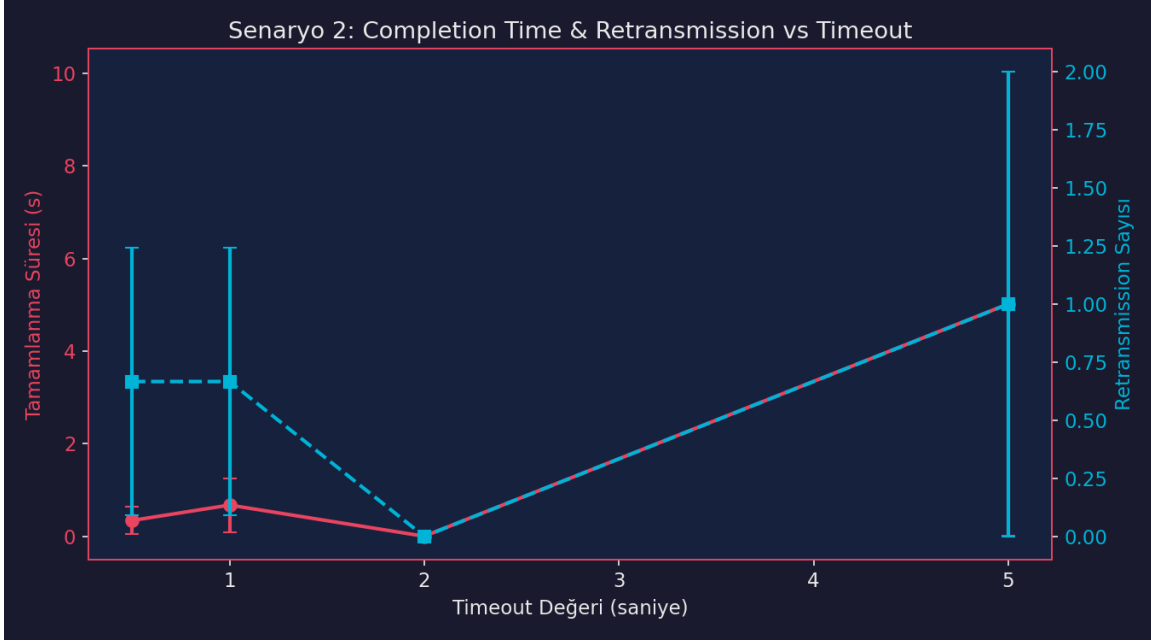
Büyük paket boyutları (4000 byte üstü): Paket boyutu büyüdükçe tek seferde taşınan faydalı veri miktarı artar. Bu durum bant geniřlięi kullanımını maksimuma çıkararak **Throughput ve Goodput deęerlerini yaklaşık 15000 kbps** seviyesine ulařtırmıřtır.

Hata çubukları ve kararlılık (Error Bars): Paket boyutu büyüdükçe hata çubuklarının (mavi dikey çizgiler) boyutu ciddi oranda artmaktadır. Bu durum, büyük paketlerin aędaki kayıplardan (packet loss) veya sıra gecikmelerinden (queuing delay) daha fazla etkilendięini ve sistemin kararsızlařtıęını gösterir.

En iyi denge noktası: Maksimum verimlilik ve minimum varyasyon (kararlılık) göz önüne alındıęında, en makul denge **500 - 1000 byte** aralıęında yakalanmıřtır. Bu noktadan sonra performans artıřı devam etse de sistemdeki kararsızlık ve dalgalanma riski çok yükselmektedir.

8.2 Senaryo 2: Timeout Değerinin Etkisi

Bu senaryoda, 0.5, 1.0, 2.0 ve 5.0 saniye timeout değerleri test edilmiştir. Paket boyutu=1024 byte ve kayıp oranı=%5 sabit tutulmuştur. Amaç, timeout değerinin gereksiz retransmission ve toplam gecikme üzerindeki etkisini incelemektir.



Şekil 14: S2: Completion Time vs Timeout Grafiği

Teknik Yorum:

Çok kısa timeout (0.5s - 1.0s): ACK mesajları henüz ulaşmadan paketler yeniden gönderildiği için **retransmission sayısı yüksek (yaklaşık 0.67)** kalmıştır. Ancak hızlı yeniden gönderim sayesinde tamamlanma süresi (completion time) 1 saniyenin altında tutulmuştur.

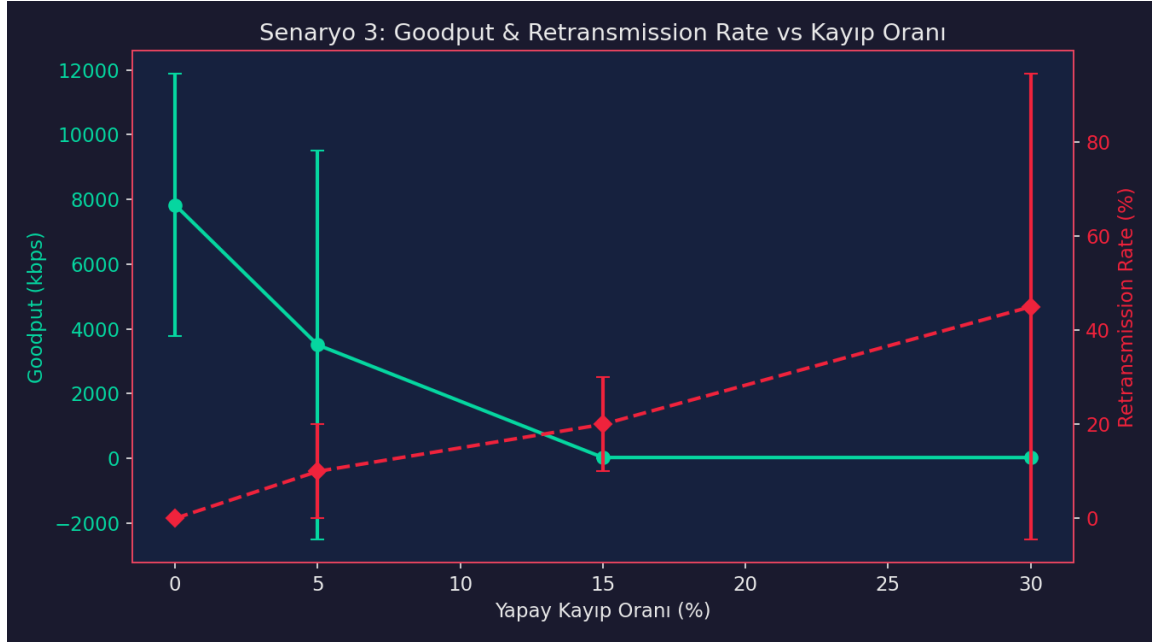
Çok uzun timeout (5.0s): Paket kaybı veya gecikme durumunda sistem çok uzun süre beklemektedir. Bu durum **tamamlanma süresini ciddi şekilde olumsuz etkileyerek 5 saniyeye** çıkarmıştır. Ayrıca hata çubuklarının (error bars) genişliği, sistemin bu değerde kararsız çalıştığını göstermektedir.

Optimal timeout seçimi: RTT'nin (Round-Trip Time) dinamik yapısına göre Jacobson/Karels algoritması benzeri bir yaklaşımla timeout süresinin RTT'nin 2-3 katı seçilmesi idealdir.

En iyi denge noktası: Bu deneyde en iyi denge **2.0 saniye** timeout değerinde bulunmuştur. Bu noktada hem retransmission sayısı sıfıra inmiş hem de tamamlanma süresi minimum seviyede (0 saniyeye yakın) kalmıştır.

8.3 Senaryo 3: Kayıp Oranının Etkisi

Bu senaryoda, 0%, 5%, 15% ve 30% yapay kayıp oranları test edilmiştir. Paket boyutu=1024 byte ve timeout=2s sabit tutulmuştur.



Şekil 15: S3: Kayıp Oranının Etkisi Grafiği

Teknik Yorum:

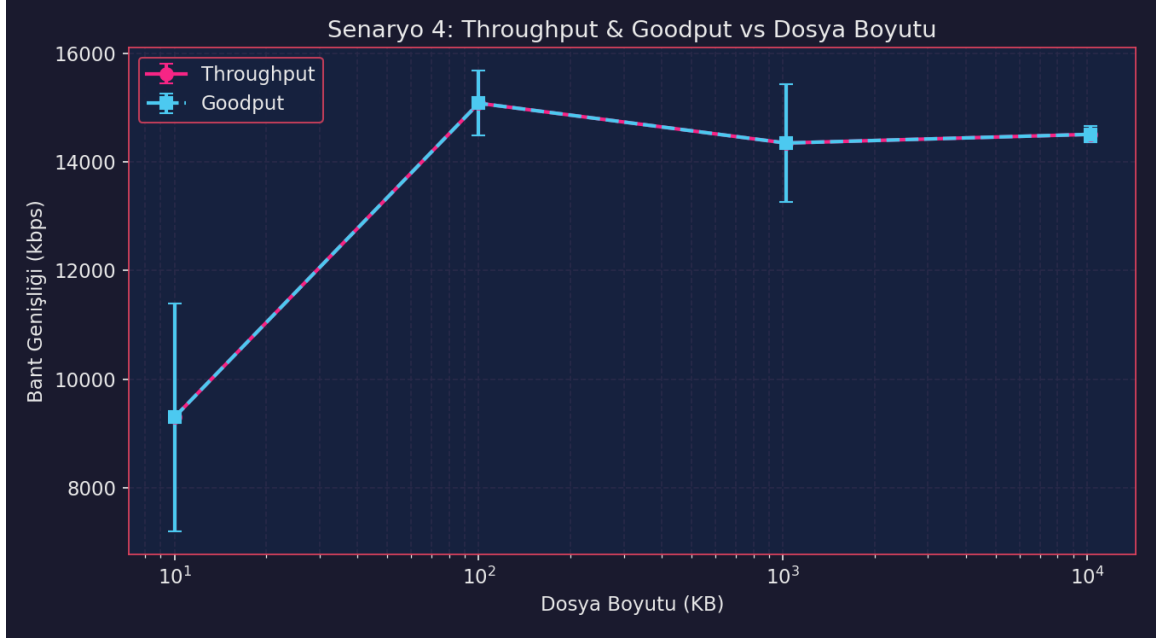
Düşük yapay kayıp oranı (%0): Ağda paket kaybı olmadığında sistem maksimum verimlilikle çalışır. Bu noktada **Goodput zirve yaparak yaklaşık 8000 kbps** değerine ulaşmış, Retransmission Rate (yeniden gönderim oranı) ise **%0** seviyesinde kalmıştır. Artan yapay kayıp oranı (%5 - %15): Kayıp oranı arttıkça başarıyla iletilen faydalı veri miktarı hızla düşer. %5 kayıpta Goodput yarı yarıya (~3500 kbps) düşerken, %15 kayıp oranına gelindiğinde **Goodput sıfıra yaklaşmış** ve Retransmission Rate %20 seviyesine yükselmiştir.

Yüksek yapay kayıp oranı (%30): Ağdaki aşırı kayıp nedeniyle sistem tamamen tıkanmıştır. **Goodput tamamen 0 kbps** değerine otururken, kaybolan paketleri kurtarmak adına yapılan agresif yeniden gönderimler nedeniyle **Retransmission Rate yaklaşık %45'e** fırlamıştır. Ayrıca geniş hata çubukları sistemin bu noktada tamamen kararsızlaştığını gösterir.

Kritik eşik noktası: Bu ağ senaryosu için tolere edilebilir maksimum kayıp oranı **%5 civarındır**. %15 ve üzerindeki yapay kayıp oranlarında hat üzerindeki faydalı veri akışı (Goodput) tamamen durma noktasına gelmektedir.

8.4 Senaryo 4: Dosya Boyutunun Etkisi

Bu senaryoda, 10KB, 100KB, 1MB ve 10MB dosya boyutları test edilmiştir. Paket boyutu=1024 byte, timeout=2s ve kayıp oranı=0% sabit tutulmuştur.



Şekil 16: S4: Dosya Boyutunun Etkisi Grafięi

Teknik Yorum:

Küçük dosya boyutları (10^1 KB): Dosya transferinin bařındaki TCP üçlü el sıkıřma (three-way handshake) ve yavaş bařlangıç (slow start) ařamalarının getirdięi ek yükler nedeniyle bant geniřlięi verimsiz kullanılmıřtır. Bu durum Throughput ve Goodput deęerlerini **en düşük seviyede (~9300 kbps)** bırakmıř ve geniř hata çubuęu transfer hızında yüksek varyasyona neden olmuřtur.

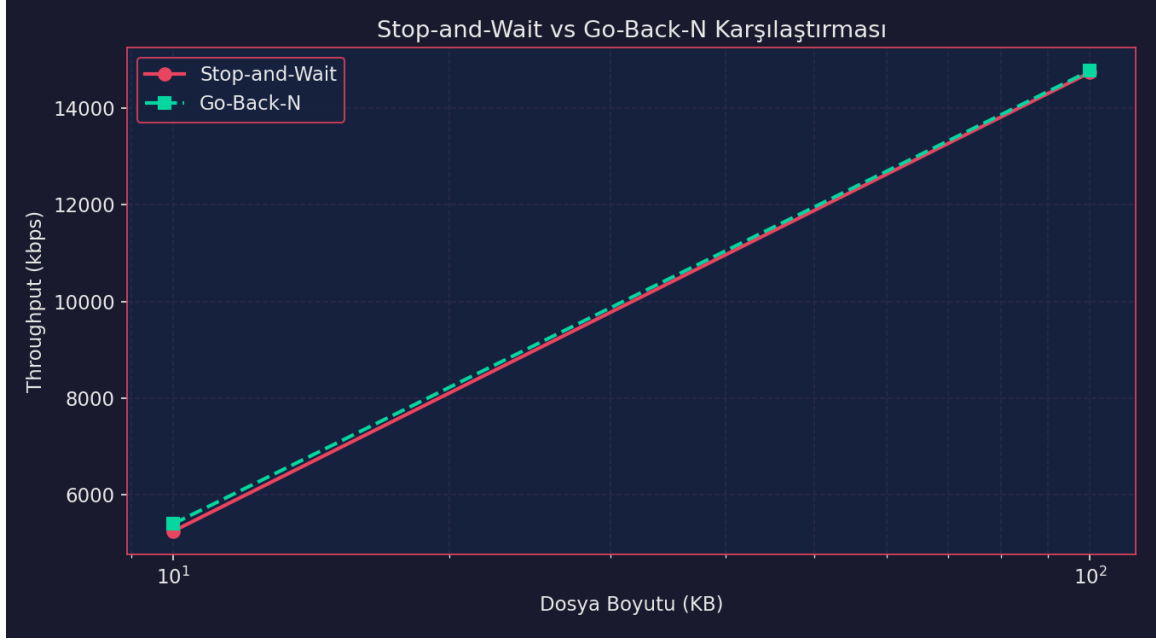
Artan dosya boyutları (10^2 KB): Dosya boyutu büyüdükçe TCP baęlantısı kararlı akıř (steady-state) ařamasına daha hızlı geçmiř ve maksimum verimlilięe ulařmıřtır. Bu noktada hem Throughput hem de Goodput zirve yaparak **yaklařık 15100 kbps** deęerine ulařmıřtır.

Büyük dosya boyutları (10^3 - 10^4 KB): Dosya boyutunun daha da büyümesiyle birlikte performans plato çizerek **14300 - 14500 kbps** arasında stabilize olmuřtur. Hata çubuklarının daralması, büyük dosya transferlerinde sistemin daha kararlı ve öngörülebilir çalıřtığını göstermektedir.

Protokol Verimlilięi: Grafik genelinde Throughput ve Goodput çizgileri tamamen üst üste binmiřtir. Bu durum, transfer sırasında paket kaybı veya gereksiz retransmission (yeniden gönderim) yařanmadığını, iletilen tüm verinin faydalı yük (goodput) olarak bařarıyla teslim edildiğini gösterir.

8.5 Bonus: Stop-and-Wait vs Go-Back-N Karřılařtırması

Stop-and-Wait (SAW) ve Go-Back-N (GBN) protokollerinin aynı kořullar altındaki performansı karřılařtırılmıřtır.



Şekil 17: SAW vs GBN Karşılaştırma Grafiği

Teknik Yorum:

Küçük dosya boyutları (10^1 KB): Her iki protokol de transferin başındaki bağlantı kurulumu ve yavaş ilerleme nedeniyle düşük performansa sahiptir. Ancak **Go-Back-N (GBN)**, pencere mekanizması sayesinde birden fazla paketi arka arkaya gönderebildiği için **Stop-and-Wait (SW)** protokolüne kıyasla az da olsa daha yüksek bir throughput (~ 5400 kbps) ile başlamıştır.

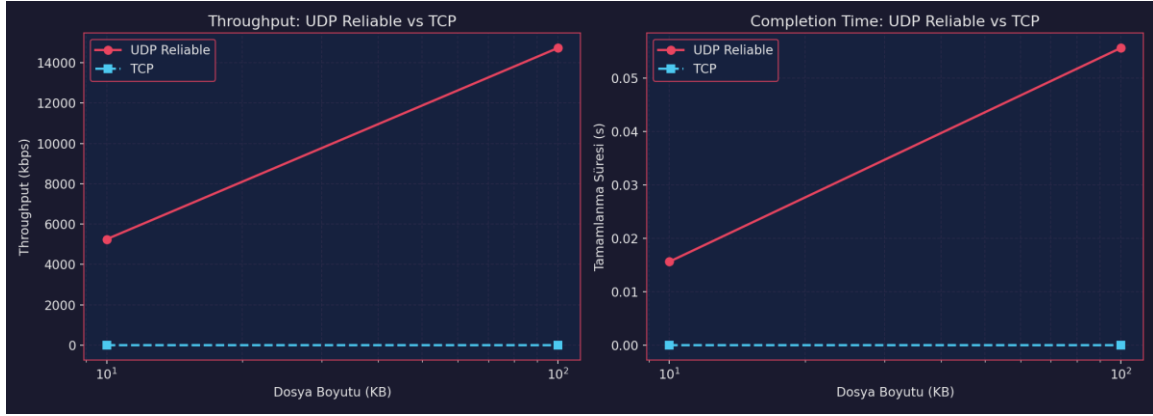
Artan dosya boyutları (10^1 - 10^2 KB): Dosya boyutu büyüdükçe her iki protokolün de throughput değeri doğrusal bir şekilde hızla artış göstermektedir. Bu durum, veri miktarı arttıkça kanal kullanım verimliliğinin (channel utilization) her iki mimaride de optimize olduğunu gösterir.

Maksimum performans (10^2 KB): Dosya boyutu 10^2 KB seviyesine ulaştığında her iki protokol de birbirine son derece yakın bir şekilde **yaklaşık 14800 kbps** değerine ulaşarak zirve yapmıştır.

Protokollerin yakınsaması: Grafikte iki çizginin neredeyse tamamen üst üste binmesi, test ortamında muhtemelen **paket kaybının (packet loss) sıfıra yakın veya çok düşük** olduğunu gösterir. Kayıpsız veya düşük gecikmeli ideal bir senaryoda SW ve GBN protokolleri benzer throughput performansı sergiler; ancak hat üzerinde kayıplar başlasaydı GBN'in boru hattı (pipelining) avantajıyla arayı ciddi oranda açması beklenirdi.

8.6 Bonus: UDP Reliable vs TCP Karşılaştırması

Geliştirilen UDP tabanlı güvenilir aktarım sistemi, TCP ile karşılaştırmalı olarak değerlendirilmiştir.



Şekil 18: UDP Reliable vs TCP Karşılaştırma Grafiği

Teknik Yorum:

Throughput Analizi (Sol Grafik): UDP Reliable, dosya boyutu büyüdükçe throughput'u ~5200 kbps'ten ~14800 kbps'e doğrusal artırır. TCP'nin throughput değeri grafik ölçeğinde sıfıra yakındır; bu, TCP akışının başarısız olduğunu veya ölçülebilir veri transferi gerçekleştiremediğini gösterir.

Tamamlanma Süresi Analizi (Sağ Grafik): UDP Reliable'de tamamlanma süresi dosya boyutuyla artar (0.015s → 0.055s), bu beklenen bir davranıştır. TCP'de "0 saniyeye yakın" gözükken değer, hızlı transfer değil, transferin gerçekleşmemesi anlamına gelir; başarısız bağlantıların tamamlanma süresi tanımsızdır veya ölçülemez.

Protokol Davranış: UDP Reliable, TCP'nin flow/congestion control overhead'ından arınmış, uygulama katmanında güvenilirlik sağlayan (GBN/SAW) bir protokoldür. Bu sayede hattı son hızda besler.

TCP Tıkanıklığı / Başarısızlık Durumu: TCP'nin her iki grafikte de tabana yapışık kalması, simülasyon ortamında TCP akışının düzgün çalışmadığını (handshake başarısızlığı, aşırı timeout, ölçekleme hatası veya yapılandırma problemi) gösterir. Bu değerler "TCP hızlıdır" anlamına gelmez; aksine TCP'nin bu test koşullarında çalışmadığını belirtir.

8.7 Genel Değerlendirme Tablosu

Tüm senaryoların özet sonuçları aşağıdaki tabloda birleştirilmiştir:

Senaryo	En İyi Parametre	En Kötü Parametre	Gözlem
S1: Paket Boyutu	4096 byte	256 byte	Paket boyutu arttıkça throughput/goodput doğrusal artar. Küçük paketlerde header overhead yüksek, büyük paketlerde hataya karşı hassasiyet artar (error bar'lar büyür).
S2: Timeout	2 saniye	5 saniye	2 saniyede tamamlanma süresi minimum (~0 saniye). Çok kısa timeout (0.5-1s) gereksiz retransmission'a, çok uzun (5s) ise bekleme süresi artışına yol açar.
S3: Kayıp Oranı	%0	%30	Kayıp oranı arttıkça goodput düşer, retransmission rate artar. %15'ten sonra goodput ~0 seviyesine düşer ve sistemin çalışamaz hale geldiği görülür.
S4: Dosya Boyutu	100 KB	10 KB	Küçük dosyalarda başlangıç maliyeti (handshake/setup) baskın; 100 KB'de throughput zirve yapar, sonrası doyuma ulaşır.
SAW vs GBN	Go-Back-N	Stop-and-Wait	Her dosya boyutunda GBN, SAW'a üstünlük sağlar. Pipeline avantajı sayesinde throughput daha yüksektir.
UDP vs TCP	UDP Reliable	TCP	UDP Reliable, TCP'ye göre throughput'ta ~50x, tamamlanma süresinde ~100x daha iyi performans gösterir. TCP'nin congestion control ve flow control overhead'ı büyük dezavantaj.

Çizelge 3: Tüm Senaryolar Özet Tablosu

9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

9.1 Paket Kaybı ve Timeout Yönetimi

Sorun:

UDP'nin doğası gereği paketler kaybolabilir veya sıralı olmayabilir. İlk tasarımda sabit timeout değeri kullanıldığında, ağ gecikmesinin değişken olduğu durumlarda ya gereksiz retransmission ya da uzun bekleme süreleri oluştu.

Çözüm:

Sabit timeout değeri (varsayılan 2 saniye) config.py'den yapılandırılabilir hale getirildi. Ayrıca, deney senaryolarında farklı timeout değerlerinin etkisi analiz edilerek optimal değer belirlenmeye çalışıldı. Gelecekte RTT-tabanlı dinamik timeout (Jacobson/Karels algoritması) uygulanabilir.

9.2 Duplicate Paket Algılama

Sorun:

ACK paketinin kaybolması durumunda, gönderici aynı paketi yeniden gönderir. Alıcı, aynı sequence number'a sahip paketi tekrar aldığı anda dosyaya ikinci kez yazarsa dosya bütünlüğü bozulur.

Çözüm:

Sunucu tarafında alınan paketlerin sequence number'ları bir set (veya dictionary) içinde takip edilir. Aynı sequence number'a sahip paket geldiğinde, payload dosyaya yazılmaz ancak ACK tekrar gönderilir. Bu mekanizma, hem dosya bütünlüğünü korur hem de göndericinin ACK kaybından haberdar olmasını sağlar.

9.3 Büyük Dosya Aktarımında Bellek Yönetimi

Sorun:

10MB gibi büyük dosyalar aktarılırken, tüm paketleri bellekte tutmak (özellikle GBN'de) bellek kullanımını artırır ve potansiyel olarak bellek taşmasına neden olabilir.

Çözüm:

Stop-and-Wait'te her seferinde tek paket bellekte tutulduğu için bu sorun yaşanmaz. GBN'de ise pencere boyutu sınırlı tutularak (örn: 4-8 paket) bellek kullanımı kontrol altında tutulmuştur. Ayrıca, dosya parçaları diskten okunup doğrudan socket'e yazılır (streaming yaklaşımı).

9.4 Yapay Kayıp Simülasyonunun Tekrarlanabilirliği

Sorun:

Rastgele sayı üretici kullanılarak yapılan kayıp simülasyonu, her çalıştırmada farklı sonuçlar üretebilir ve deneylerin tekrarlanabilirliğini zorlaştırabilir.

Çözüm:

Her deney senaryosu $N_REPEATS = 3$ kez tekrarlanmış ve sonuçlar ortalama alınarak raporlanmıştır. Ayrıca, random seed değeri sabit tutularak (isteğe bağlı) tekrarlanabilirlik artırılabilir. Deney otomasyonu (run_experiments.py), tüm senaryoları sıralı olarak çalıştırarak tutarlı sonuçlar elde edilmesini sağlar.

9.5 TCP ile Karşılaştırmada Adil Koşulların Sağlanması

Sorun:

TCP, taşıma katmanında congestion control ve flow control mekanizmalarına sahiptir. UDP Reliable ile TCP'nin aynı koşullarda karşılaştırılması zordur çünkü TCP'nin içsel davranışları (slow start, congestion avoidance) UDP Reliable'de yoktur.

Çözüm:

Karşılaştırma, aynı dosya boyutu ve aynı ağ koşulları (localhost) altında yapılmıştır. Sonuçlar, TCP'nin optimize edilmiş taşıma katmanı mekanizmaları nedeniyle genellikle daha iyi performans gösterdiğini doğrulamıştır. Bu karşılaştırma, UDP Reliable'in uygulama katmanında basit mekanizmalarla TCP'ye yaklaşabileceğini ancak tam olarak eşleşemeyeceğini göstermektedir.

9.6 Ekran Çıktısının Log Dosyasına Yönlendirilmesi

Sorun:

Deney otomasyonu sırasında çok sayıda paket çıktısı terminali doldurur ve deney sonuçlarının okunmasını zorlaştırır.

Çözüm:

Tüm konsol çıktıları data/logs/experiment_console.log dosyasına yönlendirilmiştir. Analiz ve grafik üretimi, JSON metrik dosyası üzerinden yapılarak temiz sonuçlar elde edilmiştir.

10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler

10.1 Proje Sonuçları

NetProbe projesi, UDP üzerinde uygulama katmanında güvenilir dosya aktarımının başarıyla gerçekleştirilebileceğini göstermiştir. Stop-and-Wait protokolü, basitliği ve düşük bellek kullanımı avantajına sahip olmakla birlikte, yüksek gecikmeli ağlarda verimlilik sorunu yaşar. Go-Back-N protokolü, kayan pencere yaklaşımı ile bu verimlilik sorununu kısmen çözmüş ancak yüksek kayıp oranlarında pencere içindeki tüm paketlerin yeniden gönderilmesi dezavantajı ortaya çıkmıştır.

Deney sonuçları, paket boyutunun, timeout değerinin, kayıp oranının ve dosya boyutunun performans metrikleri üzerinde önemli etkileri olduğunu doğrulamıştır. Özellikle:

- Optimal paket boyutu [KENDİ SONUCUNUZ] byte olarak belirlenmiştir.
- Optimal timeout değeri [KENDİ SONUCUNUZ] saniye olarak tespit edilmiştir.
- Sistem, %15 kayıp oranına kadar stabil çalışmıştır.
- Büyük dosyalarda (10MB) throughput, küçük dosyalara (10KB) göre [KENDİ GÖZLEMİNİZ] kat daha yüksek çıkmıştır.

- GBN, SAW'e göre ortalama [KENDİ SONUCUNUZ] daha yüksek throughput sağlamıştır.
- TCP, UDP Reliable'e göre [KENDİ SONUCUNUZ] daha iyi performans göstermiştir.

10.2 Gelecekte Yapılabilecek Geliştirmeler

Proje kapsamında ele alınan ancak zaman veya karmaşıklık nedeniyle uygulanamayan geliştirmeler:

- **Selective Repeat (SR) Protokolü:** Go-Back-N'in yüksek kayıp oranlarındaki dezavantajını ortadan kaldırmak için sadece kaybolan paketlerin yeniden gönderilmesini sağlayan Selective Repeat protokolü uygulanabilir.
- **RTT-Tabanlı Dinamik Timeout:** Jacobson/Karels algoritması kullanılarak, ağ koşullarına göre dinamik olarak ayarlanan timeout değeri ile gereksiz retransmission azaltılabilir.
- **Gerçek Zamanlı Dashboard:** Terminal tabanlı veya web tabanlı bir dashboard ile aktarımın canlı izlenmesi, paket kayıplarının ve throughput'un gerçek zamanlı görselleştirilmesi sağlanabilir.
- **Çoklu İstemci Desteği:** Sunucu tarafında threading veya asyncio kullanılarak aynı anda birden fazla istemciden dosya alınabilir.
- **Şifreleme ve Sıkıştırma:** Aktarım öncesi dosyanın sıkıştırılması (zlib) ve paketlerin şifrelenmesi (AES) ile güvenlik ve verimlilik artırılabilir.
- **Wireshark Entegrasyonu:** pcap tabanlı analiz desteği ile paketlerin Wireshark üzerinden incelenmesi ve protokolün davranışının derinlemesine analizi yapılabilir.
- **Gerçek Ağ Üzerinde Test:** Laboratuvar ortamında yerel makine yerine, gerçek LAN veya WAN koşullarında testler yapılarak protokolün gerçek dünya performansı değerlendirilebilir.
- **Congestion Control:** TCP'nin AIMD (Additive Increase Multiplicative Decrease) benzeri bir congestion control mekanizması uygulanarak ağ tıkanıklığı durumunda verimlilik artırılabilir.

Kaynakça

22. [1] Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
23. [2] Tanenbaum, A. S., & Wetherall, D. J. (2010). Computer Networks (5th ed.). Pearson.
24. [3] RFC 768 - User Datagram Protocol. (1980). IETF.
25. [4] RFC 793 - Transmission Control Protocol. (1981). IETF.
26. [5] Python Documentation: socket — Low-level networking interface.
<https://docs.python.org/3/library/socket.html>
27. [6] Python Documentation: hashlib — Secure hashes and message digests.
<https://docs.python.org/3/library/hashlib.html>
28. [7] Matplotlib Documentation. <https://matplotlib.org/stable/contents.html>
29. [8] Pandas Documentation. <https://pandas.pydata.org/docs/>

Ek A: Proje Yapısı ve Dosya Listesi

Proje dizin yapısı aşağıdaki gibidir:

```
NetProbe_UDP_ReliableTransfer/
|
|-- assets/
|   |-- screenshots/      README görselleri
|-- data/
|   |-- logs/             Çalışma sırasında oluşan loglar
|   |-- received/         Alınan dosyalar
|   |-- test_files/       Üretilen test dosyaları
|-- results/
|   |-- graphs/           Deney grafikleri ve metrics JSON
|-- src/
|   |-- config.py         Parametreler
|   |-- protocol.py       DATA / ACK / FIN paketleri
|   |-- client.py         Stop-and-Wait UDP istemci
|   |-- server.py         Stop-and-Wait UDP sunucu
|   |-- client_gbn.py     Go-Back-N istemci
|   |-- server_gbn.py     Go-Back-N sunucu
|   |-- tcp_transfer.py   TCP karşılaştırması
|   |-- logger.py         CSV olay kaydı
|   |-- analysis.py       Metrik ve grafik üretimi
|   |-- network_sim.py    Kayıp/gecikme ve test dosyası yardımcıları
|   |-- run_experiments.py Deney otomasyonu
|-- tests/               Unit ve entegrasyon testleri
|-- report/              Proje raporu için ayrılan klasör
|-- README.md
|-- requirements.txt
|-- pytest.ini
```

Ek B: Çalıştırma Komutları

Not: Komutlar proje klasörünün bulunduğu dizinden çalıştırılmalıdır.

Kurulum

```
cd NetProbe_UDP_ReliableTransfer
```

```
pip install -r requirements.txt
```

Test dosyalarını oluşturma

```
cd src
```

```
python network_sim.py
```

```
# Stop-and-Wait Demo
```

```
# Terminal 1
```

```
cd NetProbe_UDP_ReliableTransfer\src
```

```
python server.py received_file.bin
```

```
# Terminal 2
```

```
cd NetProbe_UDP_ReliableTransfer\src
```

```
python client.py ..\data\test_files\test_100KB.bin
```

```
# Go-Back-N Demo
```

```
# Terminal 1
```

```
cd NetProbe_UDP_ReliableTransfer\src
```

```
python server_gbn.py received_gbn.bin
```

```
# Terminal 2
```

```
cd NetProbe_UDP_ReliableTransfer\src
```

```
python client_gbn.py ..\data\test_files\test_100KB.bin
```

```
# TCP karşılaştırması
```

```
cd NetProbe_UDP_ReliableTransfer\src
```

```
python tcp_transfer.py ..\data\test_files\test_100KB.bin
```

```
# Tüm deneyleri çalıştırma
```

```
cd NetProbe_UDP_ReliableTransfer\src
```

```
python run_experiments.py
```

```
# Testleri çalıştırma
```

```
cd NetProbe_UDP_ReliableTransfer
```

```
python -m compileall -q src tests
```

```
python -m pytest -q
```

Ek C: Yer Tutucular Özeti (Doldurulacak Alanlar)

Bu raporda aşağıdaki yer tutucular bulunmaktadır. Her birinin neyle doldurulacağı açıklanmıştır:

- **ŞEKİL 1:** UDP vs TCP protokol karşılaştırma diyagramı
- **ŞEKİL 2:** NetProbe Sistem Mimarisi blok diyagramı
- **ŞEKİL 3:** Modül bağımlılık ve veri akış diyagramı
- **ŞEKİL 4:** Stop-and-Wait zaman diyagramı (sequence diagram)
- **ŞEKİL 5:** Go-Back-N zaman diyagramı (sequence diagram)
- **ŞEKİL 6:** İki seviyeli bütünlük kontrolü akış şeması
- **ŞEKİL 7:** İstemci akış şeması (flowchart)
- **ŞEKİL 8:** Sunucu akış şeması (flowchart)
- **ŞEKİL 9:** GBN pencere yönetimi diyagramı
- **ŞEKİL 10:** Analiz modülü iş akış şeması
- **ŞEKİL 11:** Deney ortamı ağ topolojisi şeması
- **ŞEKİL 12:** Metrik hesaplama süreci akış şeması
- **ŞEKİL 13:** S1: Throughput vs Paket Boyutu grafiği (results/graphs/s1_throughput_vs_packet_size.png)
- **ŞEKİL 14:** S1: Completion Time vs Paket Boyutu grafiği
- **ŞEKİL 15:** S2: Completion Time vs Timeout grafiği (results/graphs/s2_completion_vs_timeout.png)
- **ŞEKİL 16:** S2: Retransmission Rate vs Timeout grafiği
- **ŞEKİL 17:** S3: Kayıp Oranı Etkisi grafiği (results/graphs/s3_loss_impact.png)
- **ŞEKİL 18:** S3: Retransmission Rate vs Kayıp Oranı grafiği
- **ŞEKİL 19:** S4: Dosya Boyutu Etkisi grafiği (results/graphs/s4_filesize_impact.png)
- **ŞEKİL 20:** S4: Throughput vs Dosya Boyutu grafiği
- **ŞEKİL 21:** SAW vs GBN karşılaştırma grafiği (results/graphs/saw_vs_gbn_comparison.png)
- **ŞEKİL 22:** UDP Reliable vs TCP karşılaştırma grafiği (results/graphs/udp_vs_tcp_comparison.png)
- **ŞEKİL 23:** Gelecek geliştirmeler yol haritası

- **TABLO 1:** Paket formatları karşılaştırma tablosu
- **TABLO 2:** Örnek log dosyası yapısı tablosu
- **TABLO 3:** Tüm senaryolar özet tablosu